FIT1058 — Course Notes

Structured to match Course Notes table of contents

Alston

August 25, 2026

Contents

1 Sets

1.1 Sets and elements

Note (Naive set theory). We work within *naive set theory*: a set is a primitive, undefined notion, and any well-specified collection of objects is assumed to form a set. This suffices for ordinary practice, but unrestricted set formation is inconsistent in general – *Russell's paradox* considers $R = \{x : x \notin x\}$, for which $R \in R \leftrightarrow R \notin R$, a contradiction. Axiomatic set theory (e.g. ZFC) resolves this by restricting which collections may form sets.

Definition 1.1 (Set)**.** A set A is an unordered collection of distinct objects, called *elements* (or *members*) of A.

Definition 1.2 (Membership)**.**

$$a \in A \iff a \text{ is an element of } A, \qquad a \notin A \iff \neg(a \in A).$$

Definition 1.3 (Set equality)**.**

$$A = B \iff \forall x\,(x \in A \leftrightarrow x \in B).$$

Definition 1.4 (Empty set)**.** $\emptyset = \{\}$ is the unique set satisfying $\forall x\,(x \notin \emptyset)$.

Definition 1.5 (Singleton, unordered pair)**.** $\{a\}$ has the sole element a; $\{a, b\}$ has elements a, b, with $\{a, b\} = \{b, a\}$.

Definition 1.6 (Standard number sets)**.**

$\mathbb{N} = \{0, 1, 2, 3, \dots\}$	(natural numbers)
$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$	(integers)
$\mathbb{Z}^+ = \{1, 2, 3, \dots\}$	(positive integers)
$\mathbb{Q} = \{p/q : p \in \mathbb{Z},\ q \in \mathbb{Z},\ q \neq 0\}$	(rationals)
$\mathbb{R}$	(reals)
$\mathbb{C}$	(complex numbers)

Note. Rosen takes $0 \in \mathbb{N}$; confirm this matches CN §1.4 before relying on it elsewhere.

Definition 1.7 (Universal set)**.** The *universal set* U is the set containing all objects under discussion in a given context; every set considered is implicitly $A \subseteq U$.

Definition 1.8 (Venn diagram)**.** A *Venn diagram* represents sets as regions enclosed by curves inside a rectangle representing U; overlapping regions represent intersections, non-overlapping regions represent disjointness.

1.2 Specifying sets

Definition 1.9 (Roster method)**.** $A = \{a_1, a_2, \dots, a_n\}$. Order and repetition do not matter: $\{1, 2, 3\} = \{3, 2, 1\} = \{1, 1, 2, 3\}$.

Definition 1.10 (Ellipsis notation)**.** $\{1, 2, 3, \dots\}$, or bounded: $\{1, 2, \dots, 100\}$.

Definition 1.11 (Set-builder notation)**.** $A = \{x \in U : P(x)\}$, equivalently $\{x \mid P(x)\}$, where P is a predicate.

Example 1.12. $\{x \in \mathbb{Z} : x^2 < 9\} = \{-2, -1, 0, 1, 2\}$.

Definition 1.13 (Interval notation, $a, b \in \mathbb{R}$, $a < b$)**.**

$$[a, b] = \{x \in \mathbb{R} : a \leq x \leq b\}, \qquad (a, b) = \{x \in \mathbb{R} : a < x < b\},$$

$$[a, b) = \{x \in \mathbb{R} : a \leq x < b\}, \qquad (a, b] = \{x \in \mathbb{R} : a < x \leq b\}.$$

1.3 Cardinality

Definition 1.14 (Cardinality)**.** For finite A, $|A|$ is the number of distinct elements of A. $|\emptyset| = 0$.

Definition 1.15 (Finite, infinite set)**.** A is *finite* if $A = \emptyset$ or there exists $n \in \mathbb{Z}^+$ and a bijection $\{1, 2, \ldots, n\} \to A$; then $|A| = n$. If no such n exists, A is *infinite*.

1.4 Sets of numbers

Definition 1.16 (Signed subsets)**.**

$$\mathbb{Z}^+ = \{1, 2, 3, \ldots\}, \quad \mathbb{Z}^- = \{-1, -2, -3, \ldots\}, \quad \mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}, \quad \mathbb{R}^- = \{x \in \mathbb{R} : x < 0\}.$$

Note. $\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R} \subseteq \mathbb{C}$.

1.5 Sets of strings

Definition 1.17 (Alphabet, string, length)**.** An *alphabet* Σ is a finite nonempty set of symbols. A *string* over Σ is a finite sequence of symbols from Σ. $|w|$ is the length of w; ε is the *empty string*, $|\varepsilon| = 0$.

Definition 1.18 (Σ^*, Σ^n)**.**

$$\Sigma^n = \{w : w \text{ over } \Sigma,\ |w| = n\}, \qquad \Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n.$$

1.6 Subsets and supersets

Definition 1.19 (Subset, superset)**.**

$$A \subseteq B \iff \forall x\,(x \in A \to x \in B), \qquad A \supseteq B \iff B \subseteq A.$$

Definition 1.20 (Proper subset)**.**

$$A \subsetneq B \iff (A \subseteq B) \wedge (A \neq B).$$

Theorem 1.21. *For every set A, $\emptyset \subseteq A$.*

Proof. By definition $\emptyset \subseteq A$ iff $\forall x\,(x \in \emptyset \to x \in A)$. Since $x \in \emptyset$ is always false, the conditional is vacuously true for every x, so the universally quantified statement holds. Hence $\emptyset \subseteq A$. ■

Theorem 1.22. *$\subseteq$ is reflexive ($A \subseteq A$) and transitive ($A \subseteq B \wedge B \subseteq C \Rightarrow A \subseteq C$).*

1.7 Minimal, minimum, maximal, maximum

Example 1.23 (Motivating example: cliques)**.** Let B be a set of people, with some pairs acquainted. A *clique* is a subset $A \subseteq B$ such that every two people in A know each other. We want to describe the "biggest" cliques – but "biggest" has two different meanings.

Definition 1.24 (Maximum, maximal, w.r.t. a family $\mathcal{F}$ of sets sharing a property)**.** Let $\mathcal{F}$ be a family of subsets of some set, all sharing a given property (e.g. "is a clique"), ordered by $\subseteq$.

- $M \in \mathcal{F}$ is a *maximum* if $S \subseteq M$ for every $S \in \mathcal{F}$ (largest possible size).
- $M \in \mathcal{F}$ is *maximal* if M is not a proper subset of any $S \in \mathcal{F}$ (cannot be enlarged while keeping the property).

Theorem 1.25. *Every maximum is maximal.*

Proof. If M is a maximum, then $S \subseteq M$ for all $S \in \mathcal{F}$, so no $S \in \mathcal{F}$ can have $M \subsetneq S$ (that would force $M \subsetneq S \subseteq M$, impossible). Hence M is maximal. ■

Note. The converse fails in general: a maximal clique need not be a maximum clique – it just cannot be enlarged, while some other, larger clique may exist elsewhere in B.

Definition 1.26 (Minimum, minimal – dual definitions)**.** Dually, for $\mathcal{F}$ ordered by $\subseteq$:

- $m \in \mathcal{F}$ is a *minimum* if $m \subseteq S$ for every $S \in \mathcal{F}$.
- $m \in \mathcal{F}$ is *minimal* if m is not a proper superset of any $S \in \mathcal{F}$ (removing anything from m destroys the property).

Every minimum is minimal; the converse fails in general.

Note (Why the distinction matters here but not for $\mathbb{R}$). For real numbers, $\leq$ is a *total order*: any two reals are comparable, so "cannot be increased" and "is the largest" coincide, and *maximum*/*maximal* are used interchangeably. But $\subseteq$ is only a *partial order*: two sets A, B can be *incomparable* ($A \not\subseteq B$ and $B \not\subseteq A$). This is exactly why a family can have several maximal elements without any single maximum.

1.8 Counting all subsets

Theorem 1.27. *For finite B with $|B| = n$, the number of subsets of B is 2^n.*

Proof. A subset is determined by choosing, independently for each of the n elements, whether to include it (2 options per element). By the product rule, the number of independent choice-sequences is $\underbrace{2 \times 2 \times \cdots \times 2}_{n} = 2^n$, and each choice-sequence corresponds to exactly one subset. ■

1.9 The power set of a set

Definition 1.28 (Power set)**.** $\mathcal{P}(A) = \{S : S \subseteq A\}$.

Theorem 1.29. $|A| = n \Rightarrow |\mathcal{P}(A)| = 2^n$.

Proof. Immediate from §1.8: $\mathcal{P}(A)$ is exactly the set of subsets of A, and there are 2^n of them. ■

Note. Holds even for $A = \emptyset$: $n = 0$ gives $2^0 = 1$, consistent with $\emptyset$ having exactly one subset, itself. $\mathcal{P}(A)$ is also written 2^A.

Example 1.30. $\emptyset$ has exactly one subset, itself, so $\mathcal{P}(\emptyset) = \{\emptyset\}$. The set $\{\emptyset\}$ has exactly two subsets, $\emptyset$ and $\{\emptyset\}$ itself, so $\mathcal{P}(\{\emptyset\}) = \{\emptyset, \{\emptyset\}\}$.

1.10 Counting subsets by size using binomial coefficients

Definition 1.31 (Binomial coefficient)**.** For $0 \le k \le n$, $\binom{n}{k}$ ("n choose k") is the number of k-element subsets of an n-element set.

Theorem 1.32 (Sum over k)**.**

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

Proof. The binomial coefficients $\binom{n}{0}, \binom{n}{1}, \ldots, \binom{n}{n}$ partition the subsets of an n-set by size, so they sum to the total subset count, 2^n (§1.8). ■

Theorem 1.33 (Symmetry)**.**

$$\binom{n}{k} = \binom{n}{n-k}.$$

Proof. Choosing k elements to include determines which $n-k$ elements are excluded, and vice versa; this is a bijection between k-subsets and $(n-k)$-subsets. ■

Theorem 1.34 (Closed form)**.**

$$\binom{n}{k} = \frac{n(n-1)(n-2)\cdots(n-k+1)}{k!} = \frac{n!}{k!(n-k)!}.$$

Proof. Count *ordered* choices of k distinct elements from n: n options for the first, $n-1$ for the second, …, $n-k+1$ for the k-th, giving $n(n-1)\cdots(n-k+1) = n!/(n-k)!$ ordered sequences. Each k-subset is counted once for every ordering of its k elements, i.e. $k!$ times, so

$$\binom{n}{k} = \frac{1}{k!} \cdot \frac{n!}{(n-k)!} = \frac{n!}{k!(n-k)!}.$$

■

Example 1.35. $\binom{n}{0} = \binom{n}{n} = 1$ (only one way to choose nothing, or everything); $\binom{n}{1} = \binom{n}{n-1} = n$.

1.11 Complement and set difference

Definition 1.36 (Set difference, complement w.r.t. U)**.**

$$A - B = \{x : x \in A \wedge x \notin B\}, \qquad \overline{A} = U - A.$$

Theorem 1.37 (De Morgan's laws)**.**

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \qquad \overline{A \cap B} = \overline{A} \cup \overline{B}.$$

Example 1.38. Prove $\overline{A \cap B} = \overline{A} \cup \overline{B}$ using set-builder notation and logical equivalences.

$$\begin{aligned}
\overline{A \cap B} &= \{x : x \notin A \cap B\} \\
&= \{x : \neg(x \in A \cap B)\} \\
&= \{x : \neg(x \in A \wedge x \in B)\} \\
&= \{x : \neg(x \in A) \vee \neg(x \in B)\} && \text{(De Morgan's law, logic)} \\
&= \{x : x \notin A \vee x \notin B\} \\
&= \{x : x \in \overline{A} \vee x \in \overline{B}\} \\
&= \overline{A} \cup \overline{B}.
\end{aligned}$$

Example 1.39. Let A, B, C be sets. Prove $\overline{A \cup (B \cap C)} = (\overline{C} \cup \overline{B}) \cap \overline{A}$ by chaining previously established identities.

$$\begin{aligned}
\overline{A \cup (B \cap C)} &= \overline{A} \cap \overline{(B \cap C)} && \text{(1st De Morgan's law)} \\
&= \overline{A} \cap (\overline{B} \cup \overline{C}) && \text{(2nd De Morgan's law)} \\
&= (\overline{B} \cup \overline{C}) \cap \overline{A} && \text{(commutativity of } \cap\text{)} \\
&= (\overline{C} \cup \overline{B}) \cap \overline{A}. && \text{(commutativity of } \cup\text{)}
\end{aligned}$$

1.12 Union and intersection

Definition 1.40 (Union, intersection)**.**

$$A \cup B = \{x : x \in A \vee x \in B\}, \qquad A \cap B = \{x : x \in A \wedge x \in B\}.$$

Definition 1.41 (Disjoint)**.** $A \cap B = \emptyset$.

Theorem 1.42 (Inclusion–exclusion)**.** $|A \cup B| = |A| + |B| - |A \cap B|$.

Example 1.43. Prove $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ by the element argument.

($\subseteq$) Let $x \in A \cap (B \cup C)$. Then $x \in A$ and ($x \in B$ or $x \in C$). If $x \in B$ then $x \in A \cap B$; if $x \in C$ then $x \in A \cap C$. Either way $x \in (A \cap B) \cup (A \cap C)$.

($\supseteq$) Let $x \in (A \cap B) \cup (A \cap C)$. Then $x \in A \cap B$ or $x \in A \cap C$; either way $x \in A$ and ($x \in B$ or $x \in C$), so $x \in A \cap (B \cup C)$.

Both directions hold, so the sets are equal.

Note (Methods for proving set identities). (i) *Element argument (double inclusion)*: show LHS $\subseteq$ RHS and RHS $\subseteq$ LHS, as above.

(ii) *Membership table*: tabulate membership (0/1) of an arbitrary x in each named set for every combination, and compare the LHS/RHS columns.

(iii) *Using established identities*: rewrite one side into the other via a chain of already-proven identities (the Set Identities table), analogous to proving logical equivalences.

Definition 1.44 (Family of sets)**.** A *family of sets* indexed by I is a collection $\{A_i\}_{i \in I}$, one set A_i for each $i \in I$; I is the *index set*.

Definition 1.45 (Generalized union and intersection, finite)**.**

$$\bigcup_{i=1}^{n} A_i = A_1 \cup A_2 \cup \cdots \cup A_n = \{x : \exists i \in \{1, \ldots, n\},\ x \in A_i\},$$

$$\bigcap_{i=1}^{n} A_i = A_1 \cap A_2 \cap \cdots \cap A_n = \{x : \forall i \in \{1, \ldots, n\},\ x \in A_i\}.$$

Definition 1.46 (Generalized union and intersection, arbitrary index set)**.** For a family $\{A_i\}_{i \in I}$,

$$\bigcup_{i \in I} A_i = \{x : \exists i \in I,\ x \in A_i\}, \qquad \bigcap_{i \in I} A_i = \{x : \forall i \in I,\ x \in A_i\}.$$

When $I = \mathbb{Z}^+$ one also writes $\bigcup_{i=1}^{\infty} A_i$, $\bigcap_{i=1}^{\infty} A_i$.

Theorem 1.47 (Set identities)**.** *For sets A, B, C with universal set U:*

$$\begin{aligned}
A \cup \emptyset = A, \quad A \cap U = A && \textit{(Identity laws)} \\
A \cup U = U, \quad A \cap \emptyset = \emptyset && \textit{(Domination laws)} \\
A \cup A = A, \quad A \cap A = A && \textit{(Idempotent laws)} \\
\overline{\overline{A}} = A && \textit{(Complementation law)} \\
A \cup B = B \cup A, \quad A \cap B = B \cap A && \textit{(Commutative laws)} \\
(A \cup B) \cup C = A \cup (B \cup C),\ (A \cap B) \cap C = A \cap (B \cap C) && \textit{(Associative laws)} \\
A \cup (B \cap C) = (A \cup B) \cap (A \cup C) && \textit{(Distributive law)} \\
A \cap (B \cup C) = (A \cap B) \cup (A \cap C) && \textit{(Distributive law)} \\
\overline{A \cup B} = \overline{A} \cap \overline{B}, \quad \overline{A \cap B} = \overline{A} \cup \overline{B} && \textit{(De Morgan's laws)} \\
A \cup (A \cap B) = A, \quad A \cap (A \cup B) = A && \textit{(Absorption laws)} \\
A \cup \overline{A} = U, \quad A \cap \overline{A} = \emptyset && \textit{(Complement laws)}
\end{aligned}$$

Definition 1.48 (Bit-string representation)**.** For finite $U = \{a_1, \ldots, a_n\}$ with a fixed ordering, $A \subseteq U$ corresponds to the bit string $b_1 b_2 \cdots b_n$ where $b_i = 1$ iff $a_i \in A$. Then $A \cup B \leftrightarrow$ bitwise OR, $A \cap B \leftrightarrow$ bitwise AND, $\overline{A} \leftrightarrow$ bitwise NOT.

1.13 Symmetric difference

Definition 1.49 (Symmetric difference)**.** $A \triangle B = \{x : x \in A \text{ or } x \in B \text{ but not both}\}$ (exclusive or).

Theorem 1.50 (Equivalent forms)**.**

$$A \triangle B = (A \setminus B) \cup (B \setminus A) = (A \cap \overline{B}) \cup (\overline{A} \cap B) = (A \cup B) \setminus (A \cap B).$$

Note. $A \triangle A = \emptyset$, and this is the only case where a symmetric difference is empty.

Theorem 1.51. $A = B \iff A \triangle B = \emptyset$.

Proof.

$A = B \iff A \subseteq B$ and $B \subseteq A \iff A \setminus B = \emptyset$ and $B \setminus A = \emptyset \iff (A \setminus B) \cup (B \setminus A) = \emptyset \iff A \triangle B = \emptyset$.

■

Theorem 1.52. $\overline{A \triangle B} = \overline{A} \triangle \overline{B}$.

Proof.

$$\overline{A \triangle B} = \overline{(A \cap \overline{B}) \cup (\overline{A} \cap B)} = \overline{(A \cap \overline{B})} \cap \overline{(\overline{A} \cap B)} = (\overline{A} \cup B) \cap (A \cup \overline{B}).$$

Expanding and simplifying (using distributivity and $A \cap \overline{A} = \emptyset$) gives $(A \cap \overline{B}) \cup (\overline{A} \cap B) = A \triangle B$, so the complement identity holds by the same characterization applied to $\overline{A}, \overline{B}$: $\overline{A} \triangle \overline{B} = (\overline{A} \cap B) \cup (A \cap \overline{B}) = A \triangle B = \overline{A \triangle B}$. ■

1.14 Cartesian product

Definition 1.53 (Ordered pair)**.** (a, b) collects a, b in order; $(a, b) = (c, d) \iff a = c \wedge b = d$.

Definition 1.54 (Cartesian product)**.** $A \times B = \{(a, b) : a \in A,\ b \in B\}$.

Theorem 1.55. $|A \times B| = |A| \cdot |B|$ *for finite* A, B.

Proof. There are $|A|$ choices for the first coordinate, and independently $|B|$ choices for the second, so by the product rule there are $|A| \cdot |B|$ pairs. ■

Definition 1.56 (n-tuple, generalized Cartesian product)**.** For sets $A_1, \ldots, A_n$:

$$A_1 \times A_2 \times \cdots \times A_n = \{(a_1, \ldots, a_n) : a_i \in A_i,\ i = 1, \ldots, n\},$$

with equality of n-tuples componentwise: $(a_1, \ldots, a_n) = (b_1, \ldots, b_n) \iff a_i = b_i\ \forall i$. If finite, $|A_1 \times \cdots \times A_n| = |A_1| \cdots |A_n|$.

Definition 1.57 (Power notation)**.** For a set A and $n \in \mathbb{N}$, $A^n = A \times A \times \cdots \times A$ (n factors); $A^0 = \{\emptyset\text{-tuple}\}$ is conventionally treated as a single-element set, $A^1 = A$. If A is finite, $|A^n| = |A|^n$.

Note. When A is an alphabet, A^n is identified with strings of length n (§1.5): e.g. $\{0, 1\}^3 = \{000, 001, \ldots, 111\}$. Also $\mathbb{R}^n = \mathbb{R} \times \cdots \times \mathbb{R}$ is the set of coordinates in n-dimensional space.

Theorem 1.58. $|A_1 \times \cdots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$ *for finite* A_i.

1.15 Partitions

Definition 1.59 (Partition)**.** A *partition* of A is a set of pairwise disjoint, nonempty subsets of A (called *parts*, *blocks*, or *cells*) whose union is A.

Note (Equivalent characterization). A collection of nonempty subsets of A is a partition of A iff every element of A belongs to *exactly one* part.

Example 1.60. For $A = \{a, b, c\}$: $\{\{a, c\}, \{b\}\}$ and $\{\{a\}, \{b\}, \{c\}\}$ are partitions; $\{\{a, b\}, \{c\}, \emptyset\}$ fails (empty part), $\{\{a, b\}, \{b, c\}\}$ fails (not disjoint), $\{\{a\}, \{b\}\}$ fails (union $\neq A$, misses c).

Definition 1.61 (Coarsest and finest partitions)**.** For any A: the *coarsest* partition is $\{A\}$ (one part, everything together); the *finest* partition is $\{\{a\} : a \in A\}$ (one singleton part per element). If $|A| = 1$ these coincide; otherwise they are the two extremes, with every other partition of A intermediate between them.

Note (Connection to equivalence relations (Rosen §9.5, previewed here)). Partitions are closely tied to *equivalence relations* (reflexive, symmetric, transitive relations – CN §2.16, Rosen §9.5): every equivalence relation R on A partitions A into *equivalence classes* $[a]_R = \{x \in A : x\,R\,a\}$, and conversely every partition of A arises this way from exactly one equivalence relation (namely, $x\,R\,y \iff x, y$ lie in the same part). We revisit this correspondence formally once binary relations are covered.

2 Functions

2.1 Definitions

Definition 2.1 (Function)**.** A function f consists of: a set called the *domain*; a set called the *codomain*; and a rule assigning, to each x in the domain, a unique value $f(x)$ in the codomain.

Definition 2.2 (Rosen's phrasing)**.** Let A, B be nonempty sets. A *function* f from A to B is an assignment of exactly one element of B to each element of A. We write $f(a) = b$ if b is the element assigned to a, and $f : A \to B$.

Note. x is the *argument* (parameter, input); $f(x)$ is the *value* (output).

Definition 2.3 (Equality of functions)**.** $f = g$ iff $\mathrm{dom}(f) = \mathrm{dom}(g)$, their codomains agree, and $f(x) = g(x)$ for every $x \in \mathrm{dom}(f)$.

Example 2.4. Let $\mathrm{CubeSum4}_{\mathbb{Z}}(w, x, y, z) = w^3 + x^3 + y^3 + z^3$ on $\mathrm{dom} = \mathbb{Z}^4$, and $\mathrm{CubeSum4}_{\mathbb{R}}$ the same rule on $\mathrm{dom} = \mathbb{R}^4$. Despite the identical rule, these are *different* functions, since equality of functions requires equal domains.

2.1.1 Domain

Definition 2.5 (Domain)**.** The *domain* of f, written $\mathrm{dom}(f)$, is exactly the set of valid arguments: $f(x)$ must be defined for every $x \in \mathrm{dom}(f)$, and undefined for every $x \notin \mathrm{dom}(f)$.

Note (The domain as a promise). Specifying $\mathrm{dom}(f)$ commits f to working correctly on *every* member of it – no more, no less. A larger domain is a bigger promise (more work to guarantee); a smaller domain that still covers the intended use case is often preferable.

2.1.2 Codomain & co.

Definition 2.6 (Codomain)**.** The *codomain* of f is a set containing every possible value $f(x)$ for $x \in \mathrm{dom}(f)$, but is allowed to be a *loose* superset – it may contain elements never actually attained.

Definition 2.7 (Image)**.** The *image* of f is the exact set of attained values, $\{f(x) : x \in \mathrm{dom}(f)\}$ – a subset of the codomain, not necessarily proper.

Note. CN deliberately avoids the word "range", since usage is inconsistent (sometimes the image, sometimes the codomain). Rosen uses "range" for what CN calls the image.

Note (Why allow looseness at all?). The image is often hard, or even provably impossible, to compute exactly (e.g. a function whose value depends on uncomputable properties of programs), while a simple superset codomain is easy to state. So we specify a codomain we're confident covers every value, rather than insisting on the exact image.

Definition 2.8 (Image of a set, Rosen)**.** For $f : A \to B$ and $S \subseteq A$, the *image of S under f* is

$$f(S) = \{f(s) : s \in S\} \subseteq B.$$

(The image of f itself, above, is the special case $f(A)$.)

Definition 2.9 (Onto / surjective, preview)**.** If the codomain equals the image, f is *onto* (*surjective*), a *surjection*. (Formal treatment in §2.7.)

2.1.3 Rule

Definition 2.10 (Rule)**.** The *rule* of f specifies, for each $x \in \mathrm{dom}(f)$, what $f(x)$ is – it need not specify *how* to compute it (no algorithm required).

Definition 2.11 (Graph of a function)**.** The *graph* of f is $\{(x, f(x)) : x \in \mathrm{dom}(f)\}$, the set of all argument–value pairs.

Example 2.12. A function on the finite domain $\{1, 2, 3, 4\}$ can be given as a finite graph, e.g. $\{(1, a), (2, a), (3, b), (4, c)\}$ (its image is $\{a, b, c\}$, a proper subset of a larger codomain). A function on an infinite domain typically needs a rule rather than a listed graph: the squaring function on $\mathbb{Z}$ has graph $\{(x, x^2) : x \in \mathbb{Z}\}$.

2.2 Functions in computer science

2.2.1 Functions from analysis

Note. In software development, *analysis* produces a precise specification of *what* must be done – often expressible as a mathematical function. *Design* then produces an *algorithm* for computing it (*how*); *implementation* turns that algorithm into code.

2.2.2 Functions in mathematics and programming

Note. Two senses of "function" coexist: the *mathematical* sense (domain, codomain, rule – no algorithm implied), and the *programming* sense (a named block of code that computes something). Unless flagged otherwise ("Python function", "algorithmic function"), we use the mathematical sense throughout.

2.3 Notation

Definition 2.13 (Function declaration)**.**

$$f : \text{domain} \to \text{codomain}, \qquad f(x) = \text{expression in } x.$$

Equivalently, the rule may be given with a *mapping arrow*: $x \mapsto$ expression in x (note: $\mapsto$ for the rule, $\to$ for domain/codomain – do not conflate them).

Example 2.14. $f : \mathbb{R} \to \mathbb{R},\ f(x) = x^2$, equivalently $f : \mathbb{R} \to \mathbb{R},\ x \mapsto x^2$. The codomain need not be the tightest possible: $\mathbb{R}_0^+$ (the image) would also be valid, but a codomain like $\mathbb{R} \cup \{0, -\sqrt{2}, -42\}$, while technically valid, is needlessly cluttered.

2.4 Some special functions

Definition 2.15 (Empty function)**.**

$$\emptyset : \emptyset \to \emptyset$$

Empty domain, empty codomain, vacuous rule.

Definition 2.16 (Identity function)**.** For a set A:

$$i_A : A \to A, \qquad i_A(x) = x.$$

Definition 2.17 (Indicator (characteristic) function)**.** For $A \subseteq D$:

$$\chi_A : D \to \{0,1\}, \qquad \chi_A(x) = \begin{cases} 1, & x \in A; \\ 0, & x \notin A. \end{cases}$$

Depends on D, not just A: different domains give different indicator functions.

Example 2.18.

$$\chi_{A \cup B}(x) = \max\{\chi_A(x), \chi_B(x)\} \quad \text{for all } x.$$

Definition 2.19 (Constant function)**.** For a domain D and object a:

$$c_a : D \to \{a\}, \qquad c_a(x) = a.$$

Definition 2.20 (Floor and ceiling, Rosen)**.** For $x \in \mathbb{R}$:

$$\lfloor x \rfloor = \max\{n \in \mathbb{Z} : n \leq x\}, \qquad \lceil x \rceil = \min\{n \in \mathbb{Z} : n \geq x\}.$$

2.5 Functions with multiple arguments and values

Definition 2.21 (Multiple arguments via Cartesian product)**.** A function of two arguments $f(x, y)$, $x \in X, y \in Y$, value in C, is formally $f : X \times Y \to C$ – a function of *one* argument, the ordered pair (x, y), with $f(x, y)$ shorthand for $f((x, y))$. Extends to any number of arguments via $X_1 \times \cdots \times X_n$.

Note (Notation styles). $f(x, y)$ is *prefix* notation (name before arguments). *Infix* (name between, e.g. $x + y$) and *postfix* (name after) also occur.

Definition 2.22 (Multiple (tuple-valued) outputs)**.** A function returning a pair is $f : X \to Y \times Z$; its value is a single element $(y, z) \in Y \times Z$, viewable as "two outputs." Extends to any number of returned values.

2.6 Restrictions

Definition 2.23 (Restriction)**.** For $f : A \to B$ and $X \subseteq A$, the *restriction* of f to X, written $f|_X$ (or $f \restriction_X$), is

$$f|_X : X \to B, \qquad f|_X(x) = f(x) \text{ for all } x \in X.$$

Note (Why restrict?). A smaller domain means: a smaller stored representation (fewer ordered pairs to list); a smaller "promise" to keep (less testing if implemented); sometimes simpler analysis; and sometimes *stronger properties* unavailable to the original function.

Example 2.24. Let $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$. Its restriction $f|_{\mathbb{R}_0^+} : \mathbb{R}_0^+ \to \mathbb{R}$ is strictly increasing throughout its domain (unlike f, which decreases on $\mathbb{R}_0^-$), and every y in its image has a *unique* preimage $\sqrt{y}$ – whereas under f itself, each nonzero y in the image has two preimages, $\pm\sqrt{y}$. (This makes $f|_{\mathbb{R}_0^+}$ invertible, unlike f – see §2.8.)

2.7 Injections, surjections, bijections

Definition 2.25 (Injective / one-to-one)**.** $f : A \to B$ is *injective* (an *injection*, *one-to-one*) if distinct arguments always give distinct values:

$$\forall x_1, x_2 \in A, \quad x_1 \neq x_2 \implies f(x_1) \neq f(x_2).$$

Note (Rosen's contrapositive phrasing). Equivalently (contrapositive):

$$\forall x_1, x_2 \in A, \quad f(x_1) = f(x_2) \implies x_1 = x_2.$$

This is the form most useful in proofs: assume $f(x_1) = f(x_2)$ and derive $x_1 = x_2$.

Note (Injections preserve information). Every y in the image of an injection has a *unique* preimage, so knowing y determines x uniquely. A non-injective function is *lossy* (some inputs become indistinguishable from their output); an injective one is *lossless*.

Definition 2.26 (Surjective / onto)**.** $f : A \to B$ is *surjective* (a *surjection*, *onto*) if

$$\forall y \in B, \ \exists x \in A \text{ such that } f(x) = y,$$

i.e. the image equals the codomain.

Definition 2.27 (Bijective)**.** f is *bijective* (a *bijection*, *one-to-one correspondence*) if it is both injective and surjective.

Note (Permutation). A bijection $f : A \to A$ (domain = codomain) is also called a *permutation* of A – usually reserved for finite A.

Theorem 2.28 (Finite equal-size domain/codomain)**.** *Let $f : A \to B$ with A, B finite and $|A| = |B|$. Then f is injective $\iff$ f is surjective $\iff$ f is bijective.*

Proof by induction on $n = |A| = |B|$. We show: f injective $\implies$ f bijective. (The surjective case is symmetric, swapping the roles of A and B throughout.)

Base case ($n = 0$): $A = B = \emptyset$. The unique function $\emptyset \to \emptyset$ is vacuously injective and vacuously surjective, hence bijective.

Inductive step: Suppose the claim holds for all injective functions between finite sets of size $n - 1$. Let $f : A \to B$ be injective with $|A| = |B| = n \geq 1$.

Pick any $a \in A$ and let $b = f(a)$. Set $A' = A \setminus \{a\}$, $B' = B \setminus \{b\}$, and let $f' = f|_{A'}$ be the restriction of f to A'.

- f' *maps into* B': for $x \in A'$ we have $x \neq a$, so by injectivity of f, $f(x) \neq f(a) = b$; hence $f(x) \in B \setminus \{b\} = B'$.
- f' *is injective*: it is a restriction of the injective function f.
- $|A'| = |B'| = n - 1$.

By the inductive hypothesis, $f' : A' \to B'$ is bijective, in particular surjective onto B'.

Now take any $y \in B$. Either $y = b$, in which case $y = f(a)$; or $y \in B'$, in which case surjectivity of f' gives some $x \in A'$ with $f(x) = f'(x) = y$. Either way, y has a preimage under f. So f is surjective onto B, and being injective by hypothesis, f is bijective.

By induction, the claim holds for all $n \geq 0$. ■

Note. This theorem requires *finite* A, B of equal size – it fails for infinite sets (e.g. $f : \mathbb{N} \to \mathbb{N}$, $f(x) = 2x$ is injective but not surjective, despite $|\mathbb{N}| = |\mathbb{N}|$ in the infinite-cardinality sense).

Note (Proof strategies for injectivity and surjectivity). To prove $f : A \to B$ is **injective**: let $x_1, x_2 \in A$ be arbitrary with $f(x_1) = f(x_2)$, and show $x_1 = x_2$ follows (direct proof of the contrapositive form).

To prove f is **not injective**: exhibit specific $x_1 \neq x_2 \in A$ with $f(x_1) = f(x_2)$ (one counterexample suffices).

To prove f is **surjective**: let $y \in B$ be arbitrary, and *construct* (or show the existence of) some $x \in A$ with $f(x) = y$ – typically by solving $f(x) = y$ for x in terms of y and checking $x \in A$.

To prove f is **not surjective**: exhibit a specific $y \in B$ for which no $x \in A$ satisfies $f(x) = y$ (one counterexample suffices).

2.8 Inverse functions

Note (Why uniqueness can fail). Let $f : A \to B$. For a given $y \in B$, trying to go "backwards" can fail in two distinct ways:

- *Non-uniqueness*: there exist $x_1 \neq x_2 \in A$ with $f(x_1) = f(x_2) = y$.
- *Non-existence*: y lies in the codomain but not the image, so no $x \in A$ satisfies $f(x) = y$.

Both failures are ruled out exactly when f is, respectively, injective and surjective.

Definition 2.29 (Inverse function)**.** If $f : A \to B$ is such that every $y \in B$ has a *unique* $x \in A$ with $f(x) = y$, the *inverse function* $f^{-1} : B \to A$ is defined by

$$f^{-1}(y) = \text{the unique } x \in A \text{ such that } f(x) = y.$$

Equivalently, as a set of ordered pairs (the graph of f, reversed):

$$f^{-1} = \{(y, x) : (x, y) \in f\} = \{(f(x), x) : x \in A\}.$$

Theorem 2.30. *$f : A \to B$ has an inverse function if and only if f is a bijection.*

Proof. ($\Rightarrow$) If f^{-1} exists, every $y \in B$ has *some* preimage (so f is surjective) and *at most one* preimage (so f is injective, since two preimages of the same y would break the "unique x" requirement in the definition of f^{-1}). Hence f is a bijection.

($\Leftarrow$) If f is a bijection, surjectivity guarantees every $y \in B$ has *at least one* preimage, and injectivity guarantees *at most one*; together, exactly one. This unique preimage is exactly what $f^{-1}(y)$ requires, so f^{-1} is well-defined. ■

Note (When invertibility fails). Neither the `Employer` function (not injective: two computers both mapped to Harvard College Observatory) nor the squaring function $\mathbb{R} \to \mathbb{R}$ (not injective: $2^2 = (-2)^2$; also not surjective onto all of $\mathbb{R}$) is invertible.

Theorem 2.31 (f^{-1} undoes f, and vice versa)**.** *If $f : A \to B$ is a bijection, then for all $x \in A$ and all $y \in B$:*

$$f^{-1}(f(x)) = x, \qquad f(f^{-1}(y)) = y.$$

Proof. Let $x \in A$ and $y = f(x)$. By definition, $f^{-1}(y)$ is the unique element of A mapping to y under f – and x is such an element, so $f^{-1}(f(x)) = f^{-1}(y) = x$. The second identity is symmetric: apply the same argument with the roles of f, f^{-1} swapped, using that f^{-1} is itself a bijection (below). ■

Theorem 2.32. *If f is a bijection, so is f^{-1}, and $(f^{-1})^{-1} = f$.*

Proof. $f^{-1} : B \to A$ is injective and surjective by the symmetry of the defining property (unique x for each $y \Leftrightarrow$ unique y for each x, since f itself is a bijection), hence f^{-1} is a bijection and has its own inverse. That inverse must send each x back to the y with $f^{-1}(y) = x$, i.e. $y = f(x)$ – exactly f. So $(f^{-1})^{-1} = f$. ■

Definition 2.33 (Preimage of a set, Rosen)**.** For $f : A \to B$ (*not* necessarily a bijection) and $S \subseteq B$, the *preimage* (or *inverse image*) of S under f is

$$f^{-1}(S) = \{a \in A : f(a) \in S\}.$$

Note (Overloaded notation – read carefully). $f^{-1}(S)$ (preimage of a *set*, defined for *any* function f) and $f^{-1}(y)$ (the *inverse function* applied to a point, defined *only when f is a bijection*) use the same symbol f^{-1} but are different concepts. When f is a bijection, $f^{-1}(\{y\}) = \{f^{-1}(y)\}$ – the set-level and point-level notions agree – but $f^{-1}(S)$ makes sense even when no bijection (or even no inverse function) exists.

Example 2.34. For $f : \mathbb{R} \to \mathbb{R},\ f(x) = x^2$ (not a bijection, so f^{-1} as a function does not exist), the preimage of $S = [4, 9]$ is

$$f^{-1}([4, 9]) = \{x \in \mathbb{R} : x^2 \in [4, 9]\} = [-3, -2] \cup [2, 3].$$

2.9 Composition

Definition 2.35 (Composition)**.** For $f : A \to B$ and $g : B \to C$, the *composition* $g \circ f : A \to C$ is

$$(g \circ f)(x) = g(f(x)).$$

Note (Order of writing vs. order of application). $g \circ f$ is read right-to-left in application (f first, then g), matching the order inside $g(f(x))$ – but this is the *reverse* of our usual left-to-right reading order. Read carefully.

Note (Compatibility requirement). $g \circ f$ is defined only when $\text{codomain}(f) = \text{domain}(g)$: whatever f guarantees to produce must be exactly what g is guaranteed to handle. If this fails, the composition is simply undefined – not partially defined, not an error to patch around.

Theorem 2.36 (Not commutative, but associative)**.** *In general $g \circ f \neq f \circ g$ (indeed one side may not even be defined). But for $f : A \to B,\ g : B \to C,\ h : C \to D$:*

$$h \circ (g \circ f) = (h \circ g) \circ f,$$

so $h \circ g \circ f$ is unambiguous without parentheses.

Proof. Both sides are functions $A \to D$. For any $x \in A$:

$$\big(h \circ (g \circ f)\big)(x) = h\big((g \circ f)(x)\big) = h(g(f(x))) = (h \circ g)(f(x)) = \big((h \circ g) \circ f\big)(x).$$

Equal on every input, so the functions are equal. ■

Theorem 2.37 (Role of the identity)**.** *For $f : A \to A$ a bijection: $f \circ f^{-1} = i_A$ and $f^{-1} \circ f = i_A$. For any $g : C \to D$: $g \circ i_C = g$ and $i_D \circ g = g$.*

Definition 2.38 (Iterated composition)**.** For $f : A \to A$ and $n \in \mathbb{Z}^+$:

$$f^{(n)} = \underbrace{f \circ f \circ \cdots \circ f}_{n \text{ copies}}.$$

Theorem 2.39 (Composition preserves injectivity/surjectivity, Rosen)**.** *Let $f : A \to B$, $g : B \to C$.*

(i) If f, g are both injective, so is $g \circ f$.

(ii) If f, g are both surjective, so is $g \circ f$.

(iii) If f, g are both bijective, so is $g \circ f$.

Proof. (i) Suppose $(g \circ f)(x_1) = (g \circ f)(x_2)$, i.e. $g(f(x_1)) = g(f(x_2))$. Since g is injective, $f(x_1) = f(x_2)$; since f is injective, $x_1 = x_2$.

(ii) Let $z \in C$. Since g is surjective, $z = g(y)$ for some $y \in B$; since f is surjective, $y = f(x)$ for some $x \in A$. Then $(g \circ f)(x) = g(f(x)) = g(y) = z$.

(iii) Immediate from (i) and (ii). ■

2.10 Cryptosystems

Definition 2.40 (Cryptosystem)**.** A *cryptosystem* consists of a message space M, cyphertext space C, keyspace K, an *encryption function* $e : M \times K \to C$, and a *decryption function* $d : C \times K \to M$, such that for every key $k \in K$, the single-argument functions

$$e_k : M \to C,\ e_k(m) = e(m, k), \qquad d_k : C \to M,\ d_k(c) = d(c, k)$$

satisfy:

(i) e_k, d_k are bijective (onto their images), and

(ii) $d_k = e_k^{-1}$.

Note. Condition (ii) is the essential one: it forces $d_k(e_k(m)) = m$ for all $m \in M$ – decryption with k genuinely undoes encryption with k.

Example 2.41 (Shift cipher, Rosen §4.6)**.** Take $M = C = \mathbb{Z}_{26} = \{0, 1, \ldots, 25\}$ (letters as residues mod 26) and $K = \mathbb{Z}_{26}$. Define

$$e_k(p) = (p + k) \bmod 26, \qquad d_k(c) = (c - k) \bmod 26.$$

Each e_k is a bijection on $\mathbb{Z}_{26}$ with $d_k = e_k^{-1}$, so (M, C, K, e, d) is a cryptosystem. ($k = 3$ is the classical Caesar cipher.)

Definition 2.42 (Composition of cryptosystems)**.** Given $\mathcal{C} = (M, M, K, e, d)$ and $\mathcal{C}' = (M, M, K', e', d')$ (same message space), the *keyed composition* of encryption functions is

$$e' \bullet e : M \times (K \times K') \to M, \qquad (e' \bullet e)(m, (k, k')) = e'(e(m, k), k'),$$

and similarly $d \bullet d'$ for decryption. The composed cryptosystem is $\mathcal{C}' \circ \mathcal{C} = (M, M, K \times K', e' \bullet e, d \bullet d')$. For each key pair (k, k'), the resulting single-argument maps are exactly the ordinary function compositions:

$$e_{(k,k')} = e'_{k'} \circ e_k, \qquad d_{(k,k')} = d_k \circ d'_{k'}.$$

Theorem 2.43. *The composition of two cryptosystems is a cryptosystem.*

Proof. Fix $(k, k') \in K \times K'$. Since $\mathcal{C}, \mathcal{C}'$ are cryptosystems, $e_k, e'_{k'}, d_k, d'_{k'}$ are all bijections. By the composition-preserves-bijectivity theorem (§2.9), $e_{(k,k')} = e'_{k'} \circ e_k$ is a bijection. It remains to check $d_{(k,k')} = (e_{(k,k')})^{-1}$: since $(e'_{k'})^{-1} = d'_{k'}$ and $(e_k)^{-1} = d_k$, and the inverse of a composition reverses order, $(e'_{k'} \circ e_k)^{-1} = e_k^{-1} \circ (e'_{k'})^{-1} = d_k \circ d'_{k'} = d_{(k,k')}$, as required. ■

Note (Security is not guaranteed by composition). A good cryptosystem needs e_k, d_k easy to compute *with* k, but e_k hard to invert *without* k. Composing two hard-to-invert systems is often, but not always, harder still to break. Counterexample: composing two shift ciphers with keys k, k' gives $e'_{k'}(e_k(p)) = (p + k + k') \bmod 26 = e_{k+k'}(p)$ – just another shift cipher, no stronger than a single shift, since the keyspace effectively collapses.

2.11 Loose composition

Definition 2.44 (Loose composition)**.** For $f : A \to B$ and $g : C \to D$ (no requirement that $B = C$), the *loose composition* $g \mathbin{\hat{\circ}} f$ is

$$g \mathbin{\hat{\circ}} f : \{x \in A : f(x) \in C\} \to D, \qquad (g \mathbin{\hat{\circ}} f)(x) = g(f(x)).$$

Note (Always defined, but harder to use). Unlike ordinary ("tight") composition, loose composition is defined for *any* f, g – if $B \cap C = \emptyset$, it degenerates to the empty function. The cost: determining its domain requires inspecting the *rule* of f (which inputs land in C), not just f's stated codomain – so it is harder to reason about mechanically. This course uses tight composition throughout.

2.12 Counting functions

Theorem 2.45 (Counting all functions)**.** *If $|A| = m$, $|B| = n$, the number of functions $f : A \to B$ is n^m.*

Proof. Each of the m elements of A independently has n possible images in B; by the product rule, $\underbrace{n \times n \times \cdots \times n}_{m} = n^m$. ∎

Theorem 2.46 (Counting injections)**.** *If $|A| = m$, $|B| = n$, the number of injections $f : A \to B$ is*

$$n(n-1)(n-2)\cdots(n-m+1) = \frac{n!}{(n-m)!} \quad (\textit{Rosen: } P(n,m)).$$

Proof. Enumerate $A = \{a_1, \ldots, a_m\}$. a_1 has n choices in B; a_2 must avoid $f(a_1)$, so $n-1$ choices; …; a_m must avoid the $m-1$ already-used values, so $n-m+1$ choices. These choices are made in sequence, so by the product rule the total is $n(n-1)\cdots(n-m+1)$. ∎

Note. If $m > n$, the product includes a factor of 0, correctly giving 0 injections – consistent with the pigeonhole principle.

Theorem 2.47 (Counting bijections / permutations)**.** *If $|A| = |B| = n$, the number of bijections $f : A \to B$ is $n!$.*

Proof. By §2.7, for finite sets of equal size, bijective $\iff$ injective. So this is the injection count with $m = n$: $n(n-1)\cdots(n-n+1) = n!$. ∎

Note. In particular, the number of permutations of an n-element set (bijections $A \to A$) is $n!$.

2.13 Binary relations

Definition 2.48 (Binary relation)**.** A *binary relation* from A to B is a subset $R \subseteq A \times B$. We write xRy, $R(x, y)$, or $(x, y) \in R$ interchangeably. A relation *on* A satisfies $R \subseteq A \times A$.

Note (Relations as "relaxed functions")*.* A function $f : A \to B$ is exactly a relation $R \subseteq A \times B$ satisfying: for every $a \in A$, there is a *unique* $b \in B$ with $(a, b) \in R$. Dropping the uniqueness (and even the existence) requirement gives a general binary relation – so every function is a relation, but not conversely.

Definition 2.49 (Domain, range/codomain of a relation)**.** For $R \subseteq A \times B$:

$$\{x \in A : (x, y) \in R \text{ for some } y \in B\} \quad \text{(elements actually related to something)},$$

$$\{y \in B : (x, y) \in R \text{ for some } x \in A\} \quad \text{(elements actually related to)}.$$

Definition 2.50 ($R(x)$, $R^{-1}(y)$ – CN notation)**.** For $x \in A$: $R(x) = \{y \in B : xRy\}$ (a *set*, possibly empty – not a single value, unlike function notation). For $y \in B$: $R^{-1}(y) = \{x \in A : xRy\}$.

Definition 2.51 (Inverse relation)**.** For $R \subseteq A \times B$, the *inverse relation* $R^{-1} \subseteq B \times A$ is

$$yR^{-1}x \iff xRy.$$

Note. When R happens to be a function, this agrees with the inverse function of §2.8 – *but only when that function is a bijection.* For a general function f, f^{-1} as a *relation* always exists (just reverse the pairs); it is only a *function* in its own right when f is bijective.

2.14 Properties of binary relations

Definition 2.52 (Reflexive)**.** R on A is *reflexive* iff

$$\forall a \in A, \quad (a, a) \in R.$$

Definition 2.53 (Irreflexive)**.** R on A is *irreflexive* iff

$$\forall a \in A, \quad (a, a) \notin R.$$

Definition 2.54 (Symmetric)**.** R on A is *symmetric* iff

$$\forall x, y \in A, \quad (x, y) \in R \implies (y, x) \in R.$$

Definition 2.55 (Antisymmetric)**.** R on A is *antisymmetric* iff

$$\forall x, y \in A, \quad \big[(x, y) \in R \wedge (y, x) \in R\big] \implies x = y.$$

Definition 2.56 (Asymmetric)**.** R on A is *asymmetric* iff

$$\forall x, y \in A, \quad (x, y) \in R \implies (y, x) \notin R.$$

Definition 2.57 (Transitive)**.** R on A is *transitive* iff

$$\forall x, y, z \in A, \quad \big[(x, y) \in R \wedge (y, z) \in R\big] \implies (x, z) \in R.$$

Example 2.58. On $\mathbb{Z}$: $=$, $\leq$, $\geq$ are reflexive; $<$, $>$ are irreflexive. $=$ is symmetric; $<, \leq, >, \geq$ are not. $=, \leq, \geq$ are antisymmetric. $<$, $>$ are asymmetric. $=, <, \leq, >, \geq$ are all transitive; $\neq$ is not.

Theorem 2.59.

$$R \text{ asymmetric} \implies R \text{ antisymmetric} \wedge R \text{ irreflexive}.$$

The converse is false.

Proof. Suppose R is asymmetric. If $(x, y) \in R$ and $(y, x) \in R$ simultaneously held for $x \neq y$, this would contradict asymmetry directly, so antisymmetry holds. If $(a, a) \in R$ for some a, then taking $x = y = a$ in the asymmetry condition gives $(a, a) \in R \implies (a, a) \notin R$, a contradiction; so R is irreflexive.

For the converse: $\leq$ on $\mathbb{Z}$ is antisymmetric (if $x \leq y$ and $y \leq x$ then $x = y$) but not irreflexive ($a \leq a$ holds for all a), hence not asymmetric. ■

Example 2.60 (Checking all properties on a concrete relation)**.** Let $A = \{a, b, c, d\}$ and

$$R = \{(a, a), (a, c), (a, d), (b, a), (b, b), (b, c), (b, d), (c, b), (c, c), (d, b), (d, d)\}.$$

- *Reflexive*: yes, since $(a, a), (b, b), (c, c), (d, d) \in R$.
- *Symmetric*: no, since $(a, c) \in R$ but $(c, a) \notin R$.
- *Antisymmetric*: no, since $(b, c), (c, b) \in R$ with $b \neq c$.
- *Transitive*: no, since $(a, c), (c, b) \in R$ but $(a, b) \notin R$.

Theorem 2.61 (Properties under union and intersection)**.** *Let R, S be relations on A.*

$$\begin{aligned} R, S \text{ reflexive} &\implies R \cup S,\ R \cap S \text{ reflexive}, \\ R, S \text{ symmetric} &\implies R \cup S,\ R \cap S \text{ symmetric}, \\ R, S \text{ transitive} &\implies R \cap S \text{ transitive (but not necessarily } R \cup S). \end{aligned}$$

Proof. Reflexive: R, S reflexive $\implies \forall a \in A,\ (a, a) \in R \wedge (a, a) \in S \implies (a, a) \in R \cup S$ and $(a, a) \in R \cap S$.

Symmetric: If $(x, y) \in R \cup S$, then $(x, y) \in R$ or $(x, y) \in S$; symmetry of the relation containing it gives (y, x) in that same relation, hence $(y, x) \in R \cup S$. If $(x, y) \in R \cap S$, then (x, y) is in both, so by symmetry (y, x) is in both, hence $(y, x) \in R \cap S$.

Transitive ($\cap$ only): If $(x, y), (y, z) \in R \cap S$, both pairs lie in R, so transitivity of R gives $(x, z) \in R$; both pairs also lie in S, so $(x, z) \in S$; hence $(x, z) \in R \cap S$. Union fails in general: a chain $x \to y$ witnessed only by R and $y \to z$ witnessed only by S need not put (x, z) in R or in S individually. ■

2.15 Combining binary relations

Definition 2.62 (Union and intersection of relations)**.** For $R, S \subseteq A \times B$:

$$R \cup S = \{(x, y) : xRy \text{ or } xSy\}, \qquad R \cap S = \{(x, y) : xRy \text{ and } xSy\}.$$

Definition 2.63 (Composition of relations)**.** For $R \subseteq A \times B$ and $S \subseteq B \times C$, the *composition* $S \circ R \subseteq A \times C$ is

$$S \circ R = \{(x, z) : \exists y \in B \text{ such that } xRy \text{ and } ySz\}.$$

Note. When R, S happen to be functions, this is exactly function composition (§2.9). Relation composition generalizes it: no uniqueness of the intermediate y is required, and there may be several, or none.

Theorem 2.64 (Composing R with its inverse)**.** *For $R \subseteq A \times B$:*

$$(x, z) \in R^{-1} \circ R \iff \exists y \in B \text{ such that } xRy \text{ and } zRy.$$

Proof.

$$(x, z) \in R^{-1} \circ R \iff \exists y,\ xRy \wedge yR^{-1}z \iff \exists y,\ xRy \wedge zRy$$

using the defining property $yR^{-1}z \iff zRy$. ■

Note. So $R^{-1} \circ R$ relates $x, z \in A$ exactly when they share a common R-image in B; symmetrically, $R \circ R^{-1}$ relates elements of B that share a common preimage in A.

Definition 2.65 (Iterated composition of a relation with itself)**.** For R on A and $n \in \mathbb{Z}^+$:

$$R^{(n)} = \underbrace{R \circ R \circ \cdots \circ R}_{n \text{ copies}}.$$

Example 2.66 ("Six degrees of separation")**.** For the relation knows on the set of all people, knows$^{(2)}$ relates two people with a mutual acquaintance; knows$^{(6)}$ is (under this popular hypothesis) the complete relation on all people.

Definition 2.67 (Transitive closure)**.** The *transitive closure* of R on A is the unique relation R^+ on A such that, for all $x, y, z \in A$:

(i) $xRy \implies xR^+y$;

(ii) $xR^+y \wedge yRz \implies xR^+z$;

(iii) $xRy \wedge yR^+z \implies xR^+z$.

Note. Condition (i) is essential (it seeds the closure); either (ii) or (iii) alone suffices together with (i) – both are not simultaneously required.

Theorem 2.68.

$$R^+ = \bigcup_{i=1}^{\infty} R^{(i)}.$$

R^+ is always transitive.

Proof sketch. Each $R^{(i)}$ is contained in R^+: $R^{(1)} = R \subseteq R^+$ by (i), and if $R^{(i)} \subseteq R^+$, then $(x, z) \in R^{(i+1)}$ means $\exists y,\ (x, y) \in R^{(i)} \subseteq R^+$ and yRz, so $(x, z) \in R^+$ by (ii); induction gives $R^{(i)} \subseteq R^+$ for all i, hence $\bigcup_i R^{(i)} \subseteq R^+$. Conversely, $\bigcup_i R^{(i)}$ satisfies (i)-(iii) itself, and R^+ is the *unique* relation satisfying them, so equality follows. Transitivity of R^+ is then immediate: any finite chain of R^+-steps is absorbed into some $R^{(i)} \subseteq R^+$. ■

2.16 Equivalence relations

Definition 2.69 (Equivalence relation)**.** R on A is an *equivalence relation* iff R is reflexive, symmetric, and transitive.

Note (Why not antisymmetric too?). Equality is reflexive, symmetric, *and* antisymmetric. But requiring antisymmetry of a general equivalence relation would force $xRy \wedge yRx \implies x = y$ – collapsing "equivalent" into "identical," which defeats the purpose of a looser notion of sameness.

Example 2.70.
- Congruence modulo m: $x \equiv y \pmod{m} \iff m \mid (x-y)$.
- For any $f : A \to B$: $x \sim_f y \iff f(x) = f(y)$ defines an equivalence relation on A.
- The transitive closure of any reflexive, symmetric relation is an equivalence relation.
- For any relation R: $R \circ R^{-1}$ and $R^{-1} \circ R$ are equivalence relations.

Definition 2.71 (Equivalence class)**.** For equivalence relation R on A, an *equivalence class* is a nonempty $X \subseteq A$ such that:

$$\text{(i) } \forall x, y \in X,\ xRy; \qquad \text{(ii) } \nexists x \in X, z \notin X \text{ such that } xRz.$$

Equivalently, X is a *maximal* subset of A on which R holds between every pair.

Note. For $x \in A$, the equivalence class containing x is $[x]_R = \{y \in A : xRy\}$.

Theorem 2.72. *The equivalence classes of an equivalence relation R on A form a partition of A.*

Proof. **(a) Every $x \in A$ lies in some equivalence class.** Let $X = \{y \in A : xRy\} = [x]_R$.

- *X satisfies (i)*: for $y, z \in X$, xRy and xRz; by symmetry yRx, and with xRz and transitivity, yRz.
- *X satisfies (ii)*: suppose $v \in X$, $w \notin X$, vRw. Since $v \in X$: xRv; combined with vRw and transitivity, xRw, so $w \in X$ – contradiction. So no such v, w exist.
- $x \in X$ by reflexivity (xRx).

So X is an equivalence class containing x.

(b) Distinct equivalence classes are disjoint. Suppose X, Y are equivalence classes with $X \cap Y \neq \emptyset$; let $x \in X \cap Y$. We show $X = Y$ by showing $Y \setminus X = \emptyset$ and $X \setminus Y = \emptyset$.

If $y \in Y \setminus X$: since X is an equivalence class and $y \notin X$, $(x, y) \notin R$ (property (ii) for X, using $x \in X$). But $x, y \in Y$, and Y is an equivalence class, so property (i) forces $(x, y) \in R$ – contradiction. So $Y \setminus X = \emptyset$; symmetrically $X \setminus Y = \emptyset$. Hence $X = Y$.

So every element of A lies in exactly one equivalence class – the classes are nonempty (by definition), pairwise disjoint (part (b)), and cover A (part (a)): a partition. ■

2.17 Relations

Definition 2.73 (Ternary relation)**.** A *ternary relation* on sets A, B, C is a subset of $A \times B \times C$: a set of triples (x, y, z) with $x \in A,\ y \in B,\ z \in C$.

Definition 2.74 (n-ary relation)**.** An *n-ary relation* on sets $A_1, A_2, \ldots, A_n$ is a subset

$$R \subseteq A_1 \times A_2 \times \cdots \times A_n,$$

i.e. a set of n-tuples $(x_1, \ldots, x_n)$ with $x_i \in A_i$ for each i. Each A_i is the i-th *domain*; n is the *degree*. An n-ary relation is also called an *n-ary predicate*.

Example 2.75. (first name, middle name, surname) triples form a ternary relation. (day, month, year) likewise. Points in $\mathbb{R}^3$ form a ternary relation on $\mathbb{R}$.

Note (Connection to tables and databases, Rosen). An n-ary relation's tuples correspond to the *rows* of an n-column table; each domain A_i (analogous to a function's codomain – allowed to contain more than what actually appears) corresponds to a *column* or *attribute*. This correspondence is the basis of *relational databases*: a database table *is* an n-ary relation, its columns are attributes, and a *key* (or combination of attributes forming a key) is one that uniquely determines each tuple – analogous to how, in a binary relation that happens to be a function, the first coordinate uniquely determines the second.

Definition 2.76 (Database, Rosen §9.2.2)**.** A *database* is (in this model) an n-ary relation, its tuples called *records*, its domains $A_1, \dots, A_n$ called *fields* or *attributes*. A *primary key* is a domain (or combination of domains) whose value(s) uniquely determine each record – i.e. no two distinct tuples agree on it.

Definition 2.77 (Selection operator, Rosen §9.2.3)**.** For an n-ary relation R and a condition C on n-tuples, the *selection operator* s_C maps R to

$$s_C(R) = \{(a_1, \dots, a_n) \in R : C(a_1, \dots, a_n) \text{ holds}\}.$$

(Rows satisfying the condition – SQL's `WHERE`.)

Definition 2.78 (Projection, Rosen §9.2.3)**.** For $1 \le i_1 < i_2 < \cdots < i_m \le n$, the *projection* $P_{i_1,\dots,i_m}$ maps each n-tuple to the m-tuple of its selected components:

$$P_{i_1,\dots,i_m}(a_1, \dots, a_n) = (a_{i_1}, a_{i_2}, \dots, a_{i_m}).$$

(Columns kept, duplicates then removed – SQL's `SELECT`, confusingly named the opposite of the relational-algebra term.)

Note. Projection can reduce the number of tuples: distinct n-tuples that agree on all m retained components collapse to one m-tuple.

Definition 2.79 (Join, Rosen §9.2.3)**.** Let R have degree m, S have degree n, and fix $p \le m, n$. The *join* $J_p(R, S)$ is the relation of degree $m + n - p$ consisting of all $(m + n - p)$-tuples

$$(a_1, \dots, a_{m-p},\ c_1, \dots, c_p,\ b_1, \dots, b_{n-p})$$

such that $(a_1, \dots, a_{m-p}, c_1, \dots, c_p) \in R$ and $(c_1, \dots, c_p, b_1, \dots, b_{n-p}) \in S$ – i.e. tuples of R and S are merged wherever their last p / first p components agree.

Note (Applications, Rosen §9.2.4). n-ary relations underlie *data mining*: a store's transaction database (each transaction an n-ary tuple of purchased items) can be queried with selection/projection/join to find frequent itemsets, association rules, etc.

2.18 Counting relations

Theorem 2.80 (Counting binary relations)**.** *For finite A, B with $|A| = m$, $|B| = n$:*

$$\#\{\textit{binary relations from } A \textit{ to } B\} = 2^{mn}.$$

In particular, with $A = B$, $|A| = m$:

$$\#\{\textit{binary relations on } A\} = 2^{m^2}.$$

Proof. A binary relation from A to B is exactly a subset of $A \times B$, so the count equals $|\mathcal{P}(A \times B)| = 2^{|A \times B|}$ (§1.9). Since $|A \times B| = mn$ (§1.14), this is 2^{mn}. The case $A = B$ substitutes $n = m$. ■

Theorem 2.81 (Counting n-ary relations)**.** *For finite $A_1, \ldots, A_n$ with $|A_i| = m_i$:*

$$\#\{n\text{-ary relations on } A_1, \ldots, A_n\} = 2^{m_1 m_2 \cdots m_n}.$$

If all $A_i = A$ with $|A| = m$: 2^{m^n}.

Proof. Identical argument: an n-ary relation is a subset of $A_1 \times \cdots \times A_n$, of size $\prod_i m_i$ (§1.14 generalized), so the count is $2^{\prod_i m_i}$. ■

Example 2.82 (Counting relations with a given property, Rosen-style)**.** Reflexive relations on A with $|A| = m$: the m diagonal pairs (a, a) are forced *in*, leaving $m^2 - m$ off-diagonal pairs free to choose independently, so there are $2^{m^2 - m}$ reflexive relations on A.

3 Proofs

3.1 Theorems and proofs

Definition 3.1 (Theorem)**.** A *theorem* is a mathematical statement that has been proved true.

Definition 3.2 (Lemma, proposition, corollary – Rosen's terminology)**.** A *lemma* is a theorem whose main purpose is as a stepping stone in the proof of a more significant theorem. A *proposition* is a theorem of independent interest, typically less significant or easier to prove than the main theorems around it. A *corollary* is a theorem that follows almost immediately from one just proved.

Definition 3.3 (Proof)**.** A *proof* of a claim is a finite sequence of statements (*steps*), ending in the claim, where each step is one of:

- something already known: a *definition*, an *axiom* (a fundamental assumed property, e.g. $x(y+z) = xy + xz$), or a previously proved theorem;
- an *assumption*, made as a starting point;
- a *logical consequence* of earlier steps.

The final step establishes the claim from what precedes it.

Note (Verifiability). A proof must be readable sequentially – each step justified only by what comes *before* it, never by reading ahead. The end of a proof is marked by ■ (or historically "Q.E.D.", *quod erat demonstrandum*).

Theorem 3.4. *For any sets A, B: if $A \subseteq B$ then $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.*

Proof. Assume $A \subseteq B$. Let $X \in \mathcal{P}(A)$; then $X \subseteq A$ by definition of $\mathcal{P}(A)$. Since $A \subseteq B$, also $X \subseteq B$. Hence $X \in \mathcal{P}(B)$ by definition of $\mathcal{P}(B)$. So $X \in \mathcal{P}(A) \implies X \in \mathcal{P}(B)$, i.e. $\mathcal{P}(A) \subseteq \mathcal{P}(B)$. ■

Note (Annotating each step).

- "Assume $A \subseteq B$" – an *assumption* (the theorem is conditional on it).
- "Let $X \in \mathcal{P}(A)$" – a *definition*, naming an arbitrary element so we can reason about it generically.
- "$X \subseteq A$" – a *logical consequence* of the previous line and the definition of $\mathcal{P}(A)$.
- "$X \subseteq B$" – a consequence of $X \subseteq A$ and the assumption $A \subseteq B$.
- "$X \in \mathcal{P}(B)$" – a consequence of $X \subseteq B$ and the definition of $\mathcal{P}(B)$.
- Final line – restates the whole chain as the subset relation $\mathcal{P}(A) \subseteq \mathcal{P}(B)$.

Every line either restates a definition/axiom, states an assumption, or follows logically from what came strictly before – exactly the requirement above.

3.2 Logical deduction

Note (Modus ponens). The core deduction rule: if P has been established, and $P \implies Q$ has been established, then Q may be deduced. This is used constantly and rarely named explicitly, but it is the essence of every logical deduction step in a proof.

Note (Implication as a subset relation). In general, for statements P, Q, let $\hat{P}, \hat{Q}$ be the sets of situations in which P, resp. Q, holds. Then

$$P \implies Q \quad \text{corresponds to} \quad \hat{P} \subseteq \hat{Q}.$$

This generalizes the set-membership fact from §1.6: $A \subseteq B \iff \forall x\,(x \in A \implies x \in B)$ – the correspondence holds even when P, Q are not phrased as membership statements at all (as the domino example shows).

Note (Vacuous truth). If X is false, $X \implies Y$ is true regardless of Y – corresponding to $\emptyset \subseteq Y$ for any set Y (§1.6, Theorem 1: the empty set is a subset of every set).

Definition 3.5 (Converse)**.** The *converse* of $P \implies Q$ is $Q \implies P$ (equivalently written $P \impliedby Q$). An implication holding does *not* entail its converse holds: implication is not symmetric, as both examples above show ($P \Rightarrow Q$ but not $Q \Rightarrow P$; $X \Rightarrow Y$ but not $Y \Rightarrow X$).

Definition 3.6 (Biconditional)**.** When both $P \implies Q$ and its converse $Q \implies P$ hold, we write $P \iff Q$ ("P if and only if Q"; P, Q are *logically equivalent*: $\hat{P} = \hat{Q}$ as sets of situations).

Definition 3.7 (Contrapositive and inverse, Rosen)**.** For $P \implies Q$: the *contrapositive* is $\neg Q \implies \neg P$; the *inverse* is $\neg P \implies \neg Q$.

Theorem 3.8. *$P \implies Q$ is logically equivalent to its contrapositive $\neg Q \implies \neg P$. It is* not*, in general, equivalent to its converse or its inverse (which are themselves contrapositives of each other).*

Proof. Both $P \Rightarrow Q$ and $\neg Q \Rightarrow \neg P$ are false in exactly one case (P true, Q false) and true otherwise – identical truth tables, hence logically equivalent. The converse $Q \Rightarrow P$ and inverse $\neg P \Rightarrow \neg Q$ each fail in the case P false, Q true (for the converse) or P false, Q true (checking directly) – a different failure case from the original, so neither need agree with $P \Rightarrow Q$ in general; the domino example ($P \Rightarrow Q$ true, $Q \Rightarrow P$ false) is a concrete witness. ■

3.3 Proofs of existential and universal statements

Definition 3.9 (Existential statement)**.** An *existential statement* asserts $\exists x \in D,\ P(x)$ for some domain D and predicate P: that *something* with the stated property exists. It may be true or false.

Theorem 3.10. *English has a palindrome.*

Proof. 'rotator' is an English word, and it reads the same forwards and backwards. ■

Note (Proving $\exists x,\ P(x)$). A single *witness* suffices: exhibit one $x \in D$ and verify $P(x)$ holds. No need to search further once found.

Definition 3.11 (Universal statement)**.** A *universal statement* asserts $\forall x \in D,\ P(x)$: *everything* in D has the property.

Theorem 3.12. *Every English word has a vowel or a 'y'.*

Proof sketch. One could, in principle, check every English word individually ('aardvark' has a vowel; 'aardwolf' has a vowel; …; 'syzygy' has a 'y'; …) – but D here has tens of thousands of members, making case-by-case verification impractical (and hopeless for an infinite D). ■

Note (Proving $\forall x \in D,\ P(x)$). Checking every case individually only works for small, finite D – and not even then, practically. Instead: let x be an *arbitrary, generic* element of D (no properties assumed beyond $x \in D$), and show $P(x)$ follows from that alone. Since the argument uses nothing specific to the particular x chosen, it applies to *every* element of D simultaneously. This is the technique already used, e.g., in §1.6 ("let $x \in A$, show $x \in B$") and in §3.1's Theorem 13 proof ("let $X \in \mathcal{P}(A)$").

Note (Disproof is the mirror image). To *disprove* a universal statement $\forall x \in D,\ P(x)$ (i.e. prove its negation $\exists x \in D,\ \neg P(x)$), a single *counterexample* suffices – disproof of a universal is existential proof of its negation. Conversely, disproving an existential statement $\exists x \in D,\ P(x)$ requires proving the universal $\forall x \in D,\ \neg P(x)$ – generally the harder direction, since no single case can rule out all possibilities.

3.4 Finding proofs

Note (No algorithm for finding proofs). There is no systematic method for discovering proofs, for deep theoretical reasons (Gödel 1931, Church 1936, Turing 1936). Finding proofs draws on logical reasoning, familiarity with the objects involved, practice, creativity, and persistence – not a fixed recipe.

Note (Strategy: proving $A \subseteq B$).

1. Name a general member: "Let $x \in A$."
2. Unpack what the definition of A tells you about x.
3. Follow the logical consequences.
4. Aim to conclude x satisfies the definition of B.

Note (Strategy: proving $A = B$ (sets)). Prove $A \subseteq B$ and $A \supseteq B$ separately (§1.6). Each direction uses the subset strategy above.

Note (Strategy: proving $A = B$ (numbers)). If direct symbolic manipulation (algebraic rules) can transform expression A into expression B, use that. Otherwise, prove $A \leq B$ and $A \geq B$ separately – the numerical analogue of the double-inclusion technique.

3.5 Types of proofs

Note (Proof types).

- *Direct proof / proof by symbolic manipulation* (§3.6): assume P, derive Q via a direct chain of implications or equations.
- *Proof by construction* (§3.7, also called *proof by example*): exhibit a specific object and verify it works – a *constructive* existence proof, producing an explicit witness (contrast: *nonconstructive* existence proofs establish that something exists, e.g. via contradiction or a counting argument, without exhibiting it).
- *Proof by cases* (§3.8): split the domain into exhaustive cases, prove the claim in each.
- *Proof by contradiction* (§3.9): assume the negation of the claim, derive a contradiction (some statement and its negation both true).
- *Proof by contraposition*: to prove $P \implies Q$, instead prove the equivalent $\neg Q \implies \neg P$ (§3.2's contrapositive theorem) – useful when $\neg Q$ gives more to work with than P does.
- *Vacuous / trivial proof*: prove $P \implies Q$ by showing P is always false (vacuous) or Q is always true (trivial).

- *Proof by (mathematical) induction* (§3.10).

- *Uniqueness proofs*: show existence, then show any two witnesses must coincide.

This list is not exhaustive, and real proofs frequently mix several of these – e.g. a proof by cases where one case is handled by contradiction and another by direct construction.

3.6 Proof by symbolic manipulation

Note (The pattern). A chain of equations, each step justified by a known law (axiom or previously proved fact), linking the two sides of the claim.

Theorem 3.13 (Difference of two squares)**.**

$$(x-y)(x+y) = x^2 - y^2.$$

Proof.

$$\begin{aligned}(x-y)(x+y) &= x^2 + xy - yx - y^2 && \text{(distributive law)}\\ &= x^2 + xy - xy - y^2 && \text{(commutativity of multiplication)}\\ &= x^2 - y^2 && \text{(additive cancellation).}\end{aligned}$$

■

Note (Direction is arbitrary). The proof above started from the *right*-hand side and worked toward the left – equally valid as starting from the left. Pick whichever direction is more natural for the specific identity.

Note (Examples already seen in this note). This is exactly the technique behind: the De Morgan's-law proof via set-builder notation (§1.11), the chained-identity proof $\overline{A \cup (B \cap C)} = (\overline{C} \cup \overline{B}) \cap \overline{A}$ (§1.11), and the symmetric-difference identities $A \triangle B = \overline{A \triangle B}$ etc. (§1.13) – each is a sequence of equalities, each step citing a specific law.

3.7 Proof by construction

Note (Pattern). Describe a specific object precisely, and show it exists and satisfies the required conditions – a *constructive* proof of an existential statement (§3.3). Theorem 14 ('rotator' is a palindrome) is an example.

Note (Two mistakes to avoid).

- Using construction to attempt a *universal* statement: exhibiting one object with a property never establishes that *every* object has it.

- Mistaking an illustrative example for a proof: an example can clarify a proof's ideas, but is not itself a proof unless the statement being proved is genuinely existential.

Theorem 3.14 (Ramanujan's taxicab number, Rosen)**.** *There is a positive integer expressible as the sum of two positive cubes in two genuinely different ways.*

Proof.

$$1729 = 10^3 + 9^3 = 1000 + 729 = 12^3 + 1^3 = 1728 + 1.$$

Both decompositions use positive integers, and $\{10, 9\} \neq \{12, 1\}$, so these are two different ways. ■

Note. This is the smallest such number – hence its fame as the "taxicab number," from the (possibly apocryphal) anecdote of Hardy visiting Ramanujan in a taxi numbered 1729.

3.8 Proof by cases

Note (Pattern). Identify finitely many cases covering every possibility, and prove the theorem separately in each – each case-proof is a subproof. Cases may overlap (though this can signal redundant effort), but together they must be *exhaustive*: every object the theorem concerns must fall into at least one case.

Note (Typical shape). Often a theorem concerns an *infinite* set, split into a small number of cases, at least one of which still covers infinitely many objects – the reasoning within that case must be general enough to apply to all of them, not just verify a few instances.

Theorem 3.15 (Nonconstructive existence via cases, Rosen)**.** *There exist irrational numbers x, y such that x^y is rational.*

Proof. Consider $\sqrt{2}^{\sqrt{2}}$. Either it is rational or it is irrational – these are the only two (exhaustive) cases.

Case 1: $\sqrt{2}^{\sqrt{2}}$ is rational. Take $x = y = \sqrt{2}$, both irrational, with x^y rational by assumption. Done.

Case 2: $\sqrt{2}^{\sqrt{2}}$ is irrational. Take $x = \sqrt{2}^{\sqrt{2}}$ (irrational, by this case's assumption) and $y = \sqrt{2}$ (irrational). Then

$$x^y = \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^2 = 2,$$

which is rational. Done.

Either way, such x, y exist. ■

Note (Strikingly nonconstructive). This proves the statement true *without ever determining which case actually holds* – we don't know, from this proof alone, whether $\sqrt{2}^{\sqrt{2}}$ itself is rational or irrational. (It is, in fact, irrational – even transcendental, by the Gelfond–Schneider theorem – but that fact is not needed here.) This is the clearest illustration of the constructive/nonconstructive distinction from §3.5.

3.9 Proof by contradiction

Note (Pattern). Assume the *negation* of the claim; reason from it to a contradiction (some statement and its negation both derived as true); conclude the assumption was false, so the original claim holds.

Theorem 3.16 (Euclid)**.** *There are infinitely many prime numbers.*

Proof. Suppose, for contradiction, that there are only finitely many primes $p_1, p_2, \ldots, p_n$. Let

$$q = p_1 p_2 \cdots p_n + 1.$$

Since $q > p_i$ for every i, q is not itself prime, so q is composite and hence divisible by some prime. But for each p_i, dividing q by p_i leaves remainder 1 (since p_i divides $p_1 \cdots p_n$ exactly), so $p_i \nmid q$. So q is divisible by no prime at all – contradicting that q is composite. Hence the assumption was false: there are infinitely many primes. ■

Note (A useful auxiliary fact used implicitly). The argument above (and many proofs by contradiction over $\mathbb{N}$ or $\mathbb{Z}^+$) relies on: any nonempty set of natural numbers has a smallest element (well-ordering). This lets you say things like "let n be the smallest counterexample" and derive a contradiction from its minimality.

Theorem 3.17. *$\sqrt{2}$ is irrational.*

Proof. Suppose, for contradiction, that $\sqrt{2}$ is rational: $\sqrt{2} = a/b$ for integers a, b with no common factor (in lowest terms). Squaring, $2 = a^2/b^2$, so $a^2 = 2b^2$ – hence a^2 is even, hence a is even (an odd number squares to an odd number), say $a = 2c$. Substituting: $4c^2 = 2b^2$, so $b^2 = 2c^2$ – hence b^2 is even, hence b is even. But then a and b are *both* even, contradicting that a/b was in lowest terms. So the assumption was false: $\sqrt{2}$ is irrational. ■

Note. Alongside Euclid's proof, this is the other canonical proof-by-contradiction example in mathematics – both hinge on deriving a property (divisibility, in each case) that eventually contradicts an assumption baked into the setup (finiteness of the prime list; lowest-terms form of the fraction).

3.10 Proof by mathematical induction

Note (Principle of mathematical induction). To prove $S(n)$ holds for all $n \in \mathbb{N}$ (or all $n \geq n_0$):

- **Base case**: prove $S(1)$ (or $S(n_0)$).
- **Inductive step**: prove, for all k:

$$S(k) \implies S(k+1)$$

  (the assumption $S(k)$ here is the *inductive hypothesis*).

Conclude $S(n)$ holds for all n.

Note (Why this works). $S(1)$ holds (base case). Then:

$$S(1) \implies S(2), \qquad S(2) \implies S(3), \qquad S(3) \implies S(4), \qquad \dots$$

Induction packages this infinite chain of modus ponens applications into a single finite argument, rather than an informal "and so on forever."

Theorem 3.18 (Generalized De Morgan's law, CN Thm 18)**.** *For all $n \in \mathbb{N}$ and sets $A_1, \dots, A_n$:*

$$\overline{A_1 \cup A_2 \cup \cdots \cup A_n} = \overline{A_1} \cap \overline{A_2} \cap \cdots \cap \overline{A_n}.$$

Proof. Let $S(n)$ be this identity for a given n.

Base case ($n = 1$):

$$\overline{A_1} = \overline{A_1}.$$

Inductive step: assume $S(k)$, i.e.

$$\overline{A_1 \cup \cdots \cup A_k} = \overline{A_1} \cap \cdots \cap \overline{A_k}.$$

Then

$$\overline{A_1 \cup \cdots \cup A_k \cup A_{k+1}} = \overline{(A_1 \cup \cdots \cup A_k) \cup A_{k+1}} = \overline{(A_1 \cup \cdots \cup A_k)} \cap \overline{A_{k+1}}$$

by ordinary De Morgan's law (§1.11). Applying $S(k)$ to the first factor:

$$= (\overline{A_1} \cap \cdots \cap \overline{A_k}) \cap \overline{A_{k+1}} = \overline{A_1} \cap \cdots \cap \overline{A_{k+1}},$$

which is $S(k+1)$.

By induction, $S(n)$ holds for all n. ■

Theorem 3.19 (Postage stamp problem, Rosen – strong induction)**.** *Every integer $n \geq 12$ can be formed using only* 4*-cent and* 5*-cent stamps.*

Proof. We use *strong* induction.

Base cases ($n = 12, 13, 14, 15$):

$$12 = 4 + 4 + 4, \qquad 13 = 4 + 4 + 5, \qquad 14 = 4 + 5 + 5, \qquad 15 = 5 + 5 + 5.$$

Inductive step: let $k \geq 15$, and suppose every integer from 12 to k can be formed. Consider $k + 1$.

Since $k + 1 \geq 16$, we have $k + 1 - 4 \geq 12$, so by the strong inductive hypothesis, $k + 1 - 4$ can be formed from 4s and 5s. Add one more 4-cent stamp to form $k + 1$ itself.

By strong induction, every $n \geq 12$ can be formed. ■

Note (Why strong induction, not weak). The step for $k + 1$ reaches back to $k + 1 - 4$, which for small k is *not* simply k or $k - 1$ – ordinary (weak) induction, which only hands you $S(k)$, isn't enough here. Strong induction hands you $S(m)$ for *every* m from the base up to k, which is what the argument actually uses.

Theorem 3.20 (Deficient chessboard tiling, Rosen)**.** *For every $n \geq 1$, a $2^n \times 2^n$ chessboard with any* one *square removed can be exactly tiled by L-trominoes (L-shaped pieces covering* 3 *squares).*

Proof. **Base case** ($n = 1$): a 2×2 board minus one square is exactly one L-tromino.

Inductive step: assume every deficient $2^k \times 2^k$ board can be tiled. Consider a deficient $2^{k+1} \times 2^{k+1}$ board.

(i) Divide the board into four $2^k \times 2^k$ quadrants. The missing square lies in exactly one quadrant – that quadrant is itself a deficient $2^k \times 2^k$ board, tileable by the inductive hypothesis.

(ii) Place one L-tromino at the center of the board, covering the one corner cell of *each* of the other three quadrants nearest the center.

(iii) This makes each of those three quadrants a deficient $2^k \times 2^k$ board too (missing exactly the corner cell just covered), so each is tileable by the inductive hypothesis.

Together: one central tromino plus four tiled quadrants tiles the whole deficient board.

By induction, the claim holds for all $n \geq 1$. ■

Note. This is a genuinely constructive, recursive tiling algorithm hiding inside an induction proof – the inductive step doesn't just assert existence, it describes exactly how to build the larger tiling from four smaller ones.

3.11 Induction: more examples

Theorem 3.21 (Sum of the first n positive integers, CN Thm 19)**.** *For all $n \geq 1$:*

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Proof. **Base case** ($n = 1$): LHS $= 1$, RHS $= 1 \cdot 2/2 = 1$.

Inductive step: assume $1 + \cdots + k = k(k+1)/2$ for some $k \geq 1$. Then

$$1 + \cdots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = (k+1)\left(\frac{k}{2} + 1\right) = \frac{(k+1)(k+2)}{2},$$

which is the formula for $n = k + 1$.

By induction, the formula holds for all $n \geq 1$. ■

Theorem 3.22 ($2^n > n^2$ for $n \geq 5$, Rosen – inequality with nontrivial base)**.** *For every integer $n \geq 5$: $2^n > n^2$.*

Proof. **Base case** ($n = 5$): $2^5 = 32 > 25 = 5^2$.

Inductive step: let $k \geq 5$, assume $2^k > k^2$. We want $2^{k+1} > (k+1)^2$.

$$2^{k+1} = 2 \cdot 2^k > 2k^2 \qquad \text{(inductive hypothesis)}.$$

It suffices to show $2k^2 \geq (k+1)^2$ for $k \geq 5$:

$$2k^2 - (k+1)^2 = k^2 - 2k - 1 = (k-1)^2 - 2,$$

which is positive whenever $(k-1)^2 > 2$, i.e. certainly for $k \geq 5$. So $2k^2 > (k+1)^2$, hence $2^{k+1} > 2k^2 > (k+1)^2$.

By induction, $2^n > n^2$ for all $n \geq 5$. ■

Note (Why the base case matters here). The inequality is *false* for $n = 1, 2, 3, 4$ (e.g. $2^4 = 16 < 16...$ actually $2^4 = 16 = 4^2$, equality, not $>$). Getting the base case right – and matching it to where the inductive step's algebra actually works – is essential; a careless choice of base case can make the whole proof invalid even though the inductive step itself is correctly argued.

Theorem 3.23 (Bernoulli's inequality, Rosen)**.** *For all $x \geq -1$ and all integers $n \geq 0$:*

$$(1+x)^n \geq 1 + nx.$$

Proof. **Base case** ($n = 0$): $(1+x)^0 = 1 = 1 + 0 \cdot x$.

Inductive step: assume $(1+x)^k \geq 1 + kx$ for some $k \geq 0$. Since $x \geq -1$, we have $1 + x \geq 0$, so multiplying both sides of the inductive hypothesis by $(1+x)$ preserves the inequality:

$$(1+x)^{k+1} = (1+x)^k(1+x) \geq (1+kx)(1+x) = 1 + kx + x + kx^2 = 1 + (k+1)x + kx^2.$$

Since $kx^2 \geq 0$:

$$(1+x)^{k+1} \geq 1 + (k+1)x.$$

By induction, the inequality holds for all $n \geq 0$. ■

Note. The condition $x \geq -1$ is exactly what's needed for $1 + x \geq 0$, which is what allows multiplying an inequality through by $(1+x)$ without flipping its direction – a genuinely delicate step, not just bookkeeping.

3.12 Induction: extended example

Note (Setup). m job positions A ($|A| = m$), n applicants B ($|B| = n$), a *shortlist relation* $S \subseteq A \times B$.

For $X \subseteq A$, write

$$S(X) = \bigcup_{a \in X} S(a)$$

(§2.13 notation) for the combined shortlist of positions in X.

A valid assignment (every position filled, no applicant doubled) is exactly an *injection* $A \to B$ contained in S.

Theorem 3.24 (Necessity, CN Thm 20)**.** *If S contains an injection with domain A, then $|X| \leq |S(X)|$ for every $X \subseteq A$.*

Proof. An injection assigns distinct applicants to distinct positions, so the positions in X alone already require $|X|$ distinct applicants, all drawn from $S(X)$. ■

Theorem 3.25 (Sufficiency, CN Thm 21 – the converse, by induction on $|A|$)**.** *If $|X| \leq |S(X)|$ for every $X \subseteq A$, then S contains an injection with domain A.*

Proof. Induction on $m = |A|$.

Base case ($m = 1$):

$A = \{a\}$; the condition with $X = \{a\}$ gives $|S(a)| \geq 1$, so pick any $(a, b) \in S$ – this single pair is an injection with domain A.

Inductive step: assume the theorem for all relations with domain-size $\leq m$. Let $|A| = m + 1$ and suppose $|X| \leq |S(X)|$ for all $X \subseteq A$.

Case 1: every nonempty proper $X \subsetneq A$ has *strict* slack,

$$|X| \leq |S(X)| - 1.$$

Pick any $(a, b) \in S$. Restrict S to

$$T = S \cap \big((A \setminus \{a\}) \times (B \setminus \{b\})\big),$$

i.e. drop a and b. For $X \subseteq A \setminus \{a\}$, the strict slack gives $|T(X)| \geq |X|$ even after excluding b.

Since $|A \setminus \{a\}| = m$, the inductive hypothesis applies: T contains an injection g with domain $A \setminus \{a\}$.

Then $g \cup \{(a, b)\}$ is an injection with domain A, contained in S.

Case 2: some nonempty proper $X \subsetneq A$ has *exact* fit,

$$|X| = |S(X)|.$$

Apply the inductive hypothesis to S restricted to $X \times S(X)$ (domain size $|X| \leq m$): get an injection $g : X \to S(X)$.

For the rest, $A \setminus X$: for any $Z \subseteq A \setminus X$, a counting argument using $|X \cup Z| \leq |S(X \cup Z)|$ and $|S(X)| = |X|$ shows

$$|Z| \leq |S(Z) \setminus S(X)|.$$

So the relation $U = S \cap \big((A \setminus X) \times (B \setminus S(X))\big)$ satisfies the same slack condition on domain $A \setminus X$ (size $\leq m$), so by the inductive hypothesis, U contains an injection h with domain $A \setminus X$.

Since g, h have disjoint domains (X, $A \setminus X$) and disjoint codomains ($S(X)$, $B \setminus S(X)$), $g \cup h$ is an injection with domain A, contained in S.

Cases 1 and 2 are exhaustive, completing the inductive step. By induction, the theorem holds for all m. ■

Corollary 3.26 (CN Cor. 22)**.** *$S \subseteq A \times B$ contains an injection with domain $A \iff |X| \leq |S(X)|$ for every $X \subseteq A$.*

Note (Why this matters). This gives easily *verifiable* evidence in both directions: a "yes" is witnessed by the injection itself (checkable directly); a "no" is witnessed by a single offending set X with $|X| > |S(X)|$ – no need to search the whole space of possible assignments. This is a defect-form statement of *Hall's marriage theorem*.

3.13 Mathematical induction and statistical induction

Note. Statistical induction (drawing general conclusions from data, typically under randomness, tolerating some error) is unrelated to *mathematical induction* despite the shared name. Statistical induction is not logical deduction and cannot serve as a proof step; mathematical induction is a rigorous deductive technique. Don't conflate the two.

3.14 Programs and proofs

Note (The analogy, point by point).

- **Statements**: a program is a sequence of statements in a programming language; a proof is a sequence of statements in mathematical language (augmented by precise natural language).
- **Variables**: both use named references to objects; program variables have allocated memory, proof variables do not.
- **Sequential dependence**: a program statement uses only the most recently computed value of a variable, never a later one; a proof statement is justified only by *earlier* statements, never later ones.
- **Branching**: an if-statement picks a branch by a logical condition; proof by cases (§3.8) covers all situations via a finite set of logical conditions.
- **Calling other code**: a program calls other functions/programs; a proof invokes previously proved theorems.
- **Repetition**: a program repeats via loops or recursion; a proof repeats (in effect) via mathematical induction.
- **Bugs**: a program with a bug may crash on some inputs but still work on others; a proof with a genuine gap is *not a proof at all* – it must be repaired, or the claim may simply be false.

Note (Loops $\leftrightarrow$ induction, Rosen §5.5). Proving a loop computes the correct result typically uses a *loop invariant*: a statement I that (i) holds before the loop begins, and (ii) if true before an iteration, remains true after it – exactly the base case and inductive step of mathematical induction, applied to "iteration number." If I also implies the desired postcondition once the loop terminates, this establishes *partial correctness*.

Note (Recursion $\leftrightarrow$ structural induction, Rosen §5.3-5.4). Proving a recursive function correct mirrors proving a recursively-defined structure has some property: verify the function is correct on the base case(s) of the recursion, then verify that, *assuming* correctness on smaller/simpler inputs (the recursive calls), the function is correct on the current input – the direct analogue of an inductive hypothesis.

Note (Programming paradigms). Just as learning multiple programming paradigms (procedural, object-oriented, logic, functional) improves programming skill generally, learning to write rigorous proofs – a distinct "paradigm" of precise reasoning – transfers back to writing better, more carefully-reasoned programs.

4 Propositional Logic

4.1 Truth values

Definition 4.1 (Truth values)**.** Logic (classical, two-valued) is built on two *truth values*: *True* (T) and *False* (F).

Note. Physically, truth values may correspond to presence/absence of electrical current, magnetisation, a switch's on/off state, etc. – the abstraction lets us reason about logic independently of implementation. The *bit* (0 or 1) is a closely related abstraction: $F \leftrightarrow 0$, $T \leftrightarrow 1$.

4.2 Boolean variables

Definition 4.2 (Boolean / propositional variable)**.** A *Boolean variable* (or *propositional variable*) is a variable whose value is T or F.

4.3 Propositions

Definition 4.3 (Proposition, Rosen)**.** A *proposition* is a declarative sentence that is either true or false, but not both.

Example 4.4 (Propositions)**.** $1 + 1 = 2$ (true); "The earth is flat." (false); "It will rain tomorrow." (a proposition – has a definite truth value, even if unknown to us right now).

Example 4.5 (Not propositions)**.** Questions ("Come and work for us!"), nonsensical or non-declarative sentences (Lewis Carroll's "'Twas brillig..."), and self-referential sentences with no consistent truth value ("This statement is false.") are not propositions.

Note (Naming a proposition). A proposition may be named by a Boolean variable, e.g. let X be the proposition $1 + 1 = 2$; then X's truth value is T.

4.4 Logical operations

Note (The basic connectives).

Not	$\neg$
And	$\wedge$
Or	$\vee$
Implies	$\implies$
Equivalence	$\iff$

Logical operations combining propositions are also called *connectives*; in hardware they are implemented as *logic gates*.

4.5 Negation

Definition 4.6 (Negation)**.** For proposition P, $\neg P$ is true iff P is false.

P	$\neg P$
F	T
T	F

Theorem 4.7 (Double negation)**.**

$$\neg\neg P = P.$$

Note (Analogy with set complement). Negation mirrors complementation (§1.11): both give the "completely opposite" outcome, and applying either twice returns the original. If P is the proposition $x \in A$, then $\neg P$ is $x \in \overline{A}$: e.g. $\neg(\sqrt{2} \in \mathbb{Q})$ is the same as $\sqrt{2} \in \overline{\mathbb{Q}}$ (equivalently $\sqrt{2} \notin \mathbb{Q}$).

4.6 Conjunction

Definition 4.8 (Conjunction)**.** For propositions P, Q: $P \wedge Q$ ("P and Q") is true iff *both* P and Q are true.

P	Q	$P \wedge Q$
F	F	F
F	T	F
T	F	F
T	T	T

Note (Analogy with set intersection). For object x and sets A, B:

$$(x \in A) \wedge (x \in B) \iff x \in A \cap B, \qquad A \cap B = \{x : x \in A \wedge x \in B\}$$

(restating §1.12's definition using $\wedge$).

Definition 4.9 (Generalized (n-ary) conjunction, Rosen)**.** For propositions $P_1, \ldots, P_n$:

$$P_1 \wedge P_2 \wedge \cdots \wedge P_n$$

is true iff *every* P_i is true.

Example 4.10 (Translating English "and", Rosen)**.** "You can access the internet from campus only if you are a computer science major or you are not a freshman" involves nested connectives; more simply, "It is below freezing and snowing" translates directly as $P \wedge Q$ for $P =$"it is below freezing", $Q =$"it is snowing". Care is needed when English sentences bury an implicit "and" inside a single clause (e.g. "x is between 2 and 5" is really $(x > 2) \wedge (x < 5)$).

4.7 Disjunction

Definition 4.11 (Disjunction)**.** For propositions P, Q: $P \vee Q$ ("P or Q") is true iff *at least one* of P, Q is true.

P	Q	$P \vee Q$
F	F	F
F	T	T
T	F	T
T	T	T

Note (Inclusive, not exclusive). This is the *inclusive-or*: true even when *both* P, Q are true. Contrast with *exclusive-or* (§4.11), true precisely when *exactly one* holds.

Example 4.12 (English "or" is ambiguous, Rosen)**.** "Students who have taken CS100 or CS200 may take this class" – inclusive: having taken both still qualifies. "Soup or salad comes with this entree" – exclusive in ordinary restaurant use: you get one, not both, even though the sentence uses the same word "or." Mathematical $\vee$ always means *inclusive*-or unless stated otherwise; natural language is not reliable here.

Note (Analogy with set union). For object x and sets A, B:

$$(x \in A) \vee (x \in B) \iff x \in A \cup B, \qquad A \cup B = \{x : x \in A \vee x \in B\}$$

(restating §1.12's definition using $\vee$).

Note (Operator precedence so far, Rosen). Among the connectives introduced up to this point, precedence (highest to lowest) is:

$$\neg \ > \ \wedge \ > \ \vee.$$

So $\neg P \vee Q$ means $(\neg P) \vee Q$, and $P \wedge Q \vee R$ means $(P \wedge Q) \vee R$ – parenthesize explicitly whenever there's any doubt.

4.8 De Morgan's Laws

Theorem 4.13 (De Morgan's Laws for propositions)**.**

$$\neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg(P \wedge Q) = \neg P \vee \neg Q.$$

Proof of the first law, by truth table.

P	Q	$P \vee Q$	$\neg(P \vee Q)$	$\neg P$	$\neg Q$	$\neg P \wedge \neg Q$
F	F	F	T	T	T	T
F	T	T	F	T	F	F
T	F	T	F	F	T	F
T	T	T	F	F	F	F

The columns for $\neg(P \vee Q)$ and $\neg P \wedge \neg Q$ agree in every row, so the two expressions are *logically equivalent* – true or false together, for every assignment of truth values to P, Q. ■

Note (Building the table incrementally). The table is constructed left to right, each column using only earlier columns: start with all combinations of P, Q; build $P \vee Q$; negate it to get $\neg(P \vee Q)$; separately build $\neg P$ and $\neg Q$; conjoin them to get $\neg P \wedge \neg Q$. Comparing the two target columns (highlighted) completes the proof.

Note (Deriving the second law from the first). Rather than repeating the truth-table argument, the second law follows from the first by substitution: replace P, Q with $\neg P, \neg Q$ in the first law and simplify using double negation (§4.5).

Theorem 4.14 (Generalized De Morgan's law, CN Thm 23)**.** *For all n:*

$$\neg(P_1 \vee \cdots \vee P_n) = \neg P_1 \wedge \cdots \wedge \neg P_n.$$

Proof.

$$\begin{aligned} \text{LHS is true} &\iff P_1 \vee \cdots \vee P_n \text{ is false} \\ &\iff P_1, \ldots, P_n \text{ are all false} \\ &\iff \neg P_1, \ldots, \neg P_n \text{ are all true} \\ &\iff \text{RHS is true.} \end{aligned}$$

■

Note. This argument works for *every* n at once – unlike the truth-table method, which would need a separate (and infinite) family of tables, one per n.

Note (Correspondence with the set-theoretic De Morgan's laws). Compare directly with §1.11's Theorem (De Morgan's laws for sets): $\overline{A \cup B} = \overline{A} \cap \overline{B}$ and $\overline{A \cap B} = \overline{A} \cup \overline{B}$ are exactly the propositional laws above, applied to the propositions $x \in A$ and $x \in B$.

Note (Rosen: naming logical equivalence). Two compound propositions are *logically equivalent* iff their truth tables are identical in every row – written $P \equiv Q$ or $P \iff Q$ (a *tautology*, true under every assignment; formalized further in §4.12).

4.9 Implication

Definition 4.15 (Implication)**.** $P \implies Q$ ("if P then Q") is false only when P is true and Q is false.

P	Q	$P \implies Q$
F	F	T
F	T	T
T	F	F
T	T	T

Note (Rosen's phrasings). $P \implies Q$ reads: "if P then Q"; "P only if Q"; "P is sufficient for Q"; "Q is necessary for P"; "Q whenever P".

4.10 Equivalence

Definition 4.16 (Equivalence / biconditional)**.** $P \iff Q$ is true iff P, Q have the same truth value.

P	Q	$P \iff Q$
F	F	T
F	T	F
T	F	F
T	T	T

Theorem 4.17.

$$P \iff Q \quad \equiv \quad (P \implies Q) \wedge (Q \implies P).$$

Note (Rosen's phrasing). $P \iff Q$ reads: "P if and only if Q"; "P is necessary and sufficient for Q".

Note (Analogy with set equality). $x \in A \iff x \in B$ for all x is exactly $A = B$ (§1.6).

4.11 Exclusive-or

Definition 4.18 (Exclusive-or)**.** $P \oplus Q$ is true iff exactly one of P, Q is true.

P	Q	$P \oplus Q$
F	F	F
F	T	T
T	F	T
T	T	F

Theorem 4.19.

$$P \oplus Q = \neg(P \iff Q).$$

Theorem 4.20 (Generalized XOR parity, CN Thm 24)**.** *For all n and propositions $P_1, \ldots, P_n$: $P_1 \oplus \cdots \oplus P_n$ is true iff an* odd *number of the P_i are true.*

Proof. Induction on n.

Base case ($n = 1$): P_1 alone is true iff exactly 1 (odd) of it is true.

Inductive step: assume the claim for k propositions. Then

$$P_1 \oplus \cdots \oplus P_{k+1} = (P_1 \oplus \cdots \oplus P_k) \oplus P_{k+1}.$$

By the inductive hypothesis, the first factor is T iff an odd number of $P_1, \ldots, P_k$ are true, else F. Checking all four combinations of (that factor, P_{k+1}) against $\oplus$'s truth table shows the result is T exactly when the *total* count of true propositions among $P_1, \ldots, P_{k+1}$ is odd.

By induction, the claim holds for all n. ■

Note (Analogy with symmetric difference). $x \in A_1 \triangle \cdots \triangle A_n \iff (x \in A_1) \oplus \cdots \oplus (x \in A_n)$ (§1.13).

Example 4.21 (Knights and knaves, Rosen – a harder puzzle)**.** On an island, every inhabitant is either a *knight* (always tells the truth) or a *knave* (always lies). A says: "B is a knave." B says: "A and I are of opposite types."

Let $a =$ "A is a knight", $b =$ "B is a knight." A's statement, if A is a knight, must be true, and if A is a knave, must be false – i.e. $a \iff \neg b$. B's statement "A, B are opposite types" is $a \oplus b$; by the same reasoning, $b \iff (a \oplus b)$.

From $a \iff \neg b$: exactly one of a, b is true. Substitute into $b \iff (a \oplus b)$: since exactly one of a, b is true, $a \oplus b$ is true, so this forces b true, hence a false. So A is a knave, B is a knight.

4.12 Tautologies and logical equivalence

Definition 4.22 (Tautology)**.** A proposition that is true under every assignment of truth values (every row of its truth table is T).

Definition 4.23 (Logical equivalence)**.** $P \equiv Q$ (written $P = Q$) iff their truth tables are identical, iff $P \iff Q$ is a tautology.

Note (Key equivalences).

$$\neg\neg P = P, \qquad \neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg(P \wedge Q) = \neg P \vee \neg Q,$$
$$P \implies Q = \neg P \vee Q, \qquad P \iff Q = (P \implies Q) \wedge (Q \implies P),$$
$$P \oplus Q = \neg(P \iff Q) = P \iff \neg Q.$$

Theorem 4.24 (Exportation law, Rosen – harder derivation)**.**

$$(P \wedge Q) \implies R \;\equiv\; P \implies (Q \implies R).$$

Proof. Using $X \implies Y = \neg X \vee Y$ repeatedly:

$$\begin{aligned}
(P \wedge Q) \implies R &= \neg(P \wedge Q) \vee R = (\neg P \vee \neg Q) \vee R && \text{(De Morgan's)} \\
&= \neg P \vee (\neg Q \vee R) && \text{(associativity of } \vee) \\
&= \neg P \vee (Q \implies R) \\
&= P \implies (Q \implies R).
\end{aligned}$$

■

4.13 History

4.14 Distributive Laws

Theorem 4.25 (Distributive laws)**.**

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R), \qquad P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R).$$

Theorem 4.26 (Absorption laws, Rosen – derived, not axiomatic)**.**

$$P \vee (P \wedge Q) = P, \qquad P \wedge (P \vee Q) = P.$$

Proof.

$$\begin{aligned} P \vee (P \wedge Q) &= (P \wedge T) \vee (P \wedge Q) && \text{(identity: } P = P \wedge T) \\ &= P \wedge (T \vee Q) && \text{(distributive law)} \\ &= P \wedge T && \text{(domination: } T \vee Q = T) \\ &= P. \end{aligned}$$

The second identity is the dual (swap $\wedge \leftrightarrow \vee$, $T \leftrightarrow F$ throughout). ■

4.15 Laws of Boolean algebra

Note (Full table, dual pairs).

$$\neg\neg P = P$$

$$\neg T = F, \quad \neg F = T$$

$$P \wedge Q = Q \wedge P, \qquad P \vee Q = Q \vee P \quad \text{(commutative)}$$

$$(P \wedge Q) \wedge R = P \wedge (Q \wedge R), \qquad (P \vee Q) \vee R = P \vee (Q \vee R) \quad \text{(associative)}$$

$$P \wedge P = P, \qquad P \vee P = P \quad \text{(idempotent)}$$

$$P \wedge \neg P = F, \qquad P \vee \neg P = T \quad \text{(complement)}$$

$$P \wedge T = P, \qquad P \vee F = P \quad \text{(identity)}$$

$$P \wedge F = F, \qquad P \vee T = T \quad \text{(domination)}$$

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R), \qquad P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R) \quad \text{(distributive)}$$

$$\neg(P \vee Q) = \neg P \wedge \neg Q, \qquad \neg(P \wedge Q) = \neg P \vee \neg Q \quad \text{(De Morgan's)}$$

Example 4.27 (Algebraic proof, Rosen – harder chain)**.** Show $\neg(P \implies Q) \equiv P \wedge \neg Q$.

$$\begin{aligned} \neg(P \implies Q) &= \neg(\neg P \vee Q) \\ &= \neg\neg P \wedge \neg Q && \text{(De Morgan's)} \\ &= P \wedge \neg Q && \text{(double negation).} \end{aligned}$$

4.16 Disjunctive Normal Form

Definition 4.28 (Literal)**.** A logical variable, unnegated or negated once (X or $\neg X$).

Definition 4.29 (DNF)**.** A disjunction of conjunctions of literals.

Note (Constructing DNF from a truth table). For each row where the expression is T: form the conjunction of literals matching that row (X if $X = T$ in that row, $\neg X$ if $X = F$); disjoin all such conjunctions.

Example 4.30 (Majority function of 3 variables, Rosen-style – harder)**.** $\text{Maj}(X, Y, Z)$ is true iff at least two of X, Y, Z are true.

X	Y	Z	Maj
F	F	F	F
F	F	T	F
F	T	F	F
F	T	T	T
T	F	F	F
T	F	T	T
T	T	F	T
T	T	T	T

DNF (four true rows):

$$\text{Maj}(X, Y, Z) = (\neg X \wedge Y \wedge Z) \vee (X \wedge \neg Y \wedge Z) \vee (X \wedge Y \wedge \neg Z) \vee (X \wedge Y \wedge Z).$$

Note (Cost of DNF). Every expression has an equivalent DNF, but the number of parts can be as large as 2^k for k variables – exponential blow-up. DNF makes *satisfiability* easy to certify (any one part gives a satisfying assignment) but gives no efficient way to certify a *tautology*.

4.17 Conjunctive Normal Form

Definition 4.31 (CNF)**.** A conjunction of *clauses*, each a disjunction of literals.

Note (Duality with DNF, via De Morgan's). If P is in CNF, $\neg P$ – negation of a conjunction of disjunctions – becomes, via repeated De Morgan's and double negation, a disjunction of conjunctions of literals: DNF. So CNF and DNF are dual representations.

Note (Converting truth table $\rightarrow$ CNF). Negate the output column; build DNF for $\neg P$; negate that expression; apply De Morgan's to push the negation inward into CNF form.

Note (3-SAT, Rosen/CS – harder, beyond CN). When every CNF clause has exactly 3 literals, satisfiability is the classical *3-SAT* problem – the canonical NP-complete problem (Cook–Levin theorem). This is why CNF, not DNF, is the standard input format for SAT solvers: checking DNF satisfiability is trivial, but CNF satisfiability is computationally hard in general, which is exactly the structure that makes CNF expressive enough to encode real combinatorial problems.

4.18 Representing logical statements

Note (Top-down / bottom-up modelling). **Top-down**: identify the top-level conjunction of required conditions. **Bottom-up**: identify the atomic Boolean variables (simplest possible true/false assertions) before worrying about their eventual truth values.

Example 4.32 (CNF from natural-language rules)**.** Variables: one Boolean per guest. Rules:

$$(\text{Harry} \vee \text{Ron} \vee \text{Hermione} \vee \text{Ginny})$$

$$\wedge\ (\neg\text{Hagrid} \vee \text{Norberta})$$

$$\wedge\ (\neg\text{Fred} \vee \text{George}) \wedge (\text{Fred} \vee \neg\text{George})$$

$$\wedge\ (\neg\text{Voldemort} \vee \neg\text{Bellatrix}) \wedge (\neg\text{Voldemort} \vee \neg\text{Dolores}) \wedge (\neg\text{Bellatrix} \vee \neg\text{Dolores}).$$

"At least one of" $\to$ disjunction; "A only if B" $\to$ implication $\to \neg A \vee B$; "none or both" $\to$ biconditional $\to$ two implications; "no more than one of a set" $\to$ conjunction of pairwise "not both" clauses.

4.19 Statements about how many variables are true

Note (At most k of n variables true). Equivalent to: every set of $k+1$ variables has at least one false, i.e. at least one negated literal true. Conjunction over all $\binom{n}{k+1}$ subsets of size $k+1$ of the disjunction of their negations.

Note (At least k of n variables true). Equivalent to: every set of $n-k+1$ variables has at least one true. Conjunction over all $\binom{n}{n-k+1}$ subsets of the disjunction of the (unnegated) literals.

Note (Exactly k). Conjunction of the "at least k" and "at most k" expressions.

Example 4.33. At most 2 of w, x, y, z true:

$$(\neg w \vee \neg x \vee \neg y) \wedge (\neg w \vee \neg x \vee \neg z) \wedge (\neg w \vee \neg y \vee \neg z) \wedge (\neg x \vee \neg y \vee \neg z).$$

4.20 Universal sets of operations

Definition 4.34 (Universal set of operations)**.** X is *universal* if every logical expression is equivalent to one using only operations from X.

Theorem 4.35. $\{\wedge, \vee, \neg\}$ *is universal (immediate from DNF/CNF, §4.16-4.17).*

Theorem 4.36. $\{\wedge, \neg\}$ *and* $\{\vee, \neg\}$ *are each universal.*

Proof. De Morgan's laws express $\vee$ in terms of $\wedge, \neg$ (and vice versa), so either can be dropped while keeping $\neg$. ■

Note ($\{\wedge, \vee\}$ is not universal). Without $\neg$, every expression built from $\wedge, \vee$ is *monotone*: increasing any input from F to T can never make the output go from T to F. Since $\neg P$ itself is not monotone, it cannot be expressed – so $\{\wedge, \vee\}$ fails to be universal.

Definition 4.37 (Sheffer stroke (NAND), Rosen – single-operator universal set)**.**

$$P \mid Q = \neg(P \wedge Q).$$

Theorem 4.38 ($\{|\}$ alone is universal, Rosen)**.**

$$\neg P = P \mid P, \qquad P \wedge Q = (P \mid Q) \mid (P \mid Q), \qquad P \vee Q = (P \mid P) \mid (Q \mid Q).$$

Proof. Check each directly by truth table, or: $P \mid P = \neg(P \wedge P) = \neg P$; then $(P \mid Q) \mid (P \mid Q) = \neg(P \mid Q) = \neg\neg(P \wedge Q) = P \wedge Q$; and $(P \mid P) \mid (Q \mid Q) = \neg P \mid \neg Q = \neg(\neg P \wedge \neg Q) = P \vee Q$ (De Morgan's). Since $\{\wedge, \vee, \neg\}$ is universal and all three are expressible via $\mid$ alone, $\{|\}$ is universal. ■

Note. This is why NAND gates alone suffice to build any digital circuit – a genuinely important fact in computer engineering, well beyond what CN's syllabus covers here.